第7讲 算法：程序与计算系统之灵魂

李玉华

华中科技大学计算机学院

idcliyuhua@hust.edu.cn

第7讲 算法：程序与计算系统之灵魂

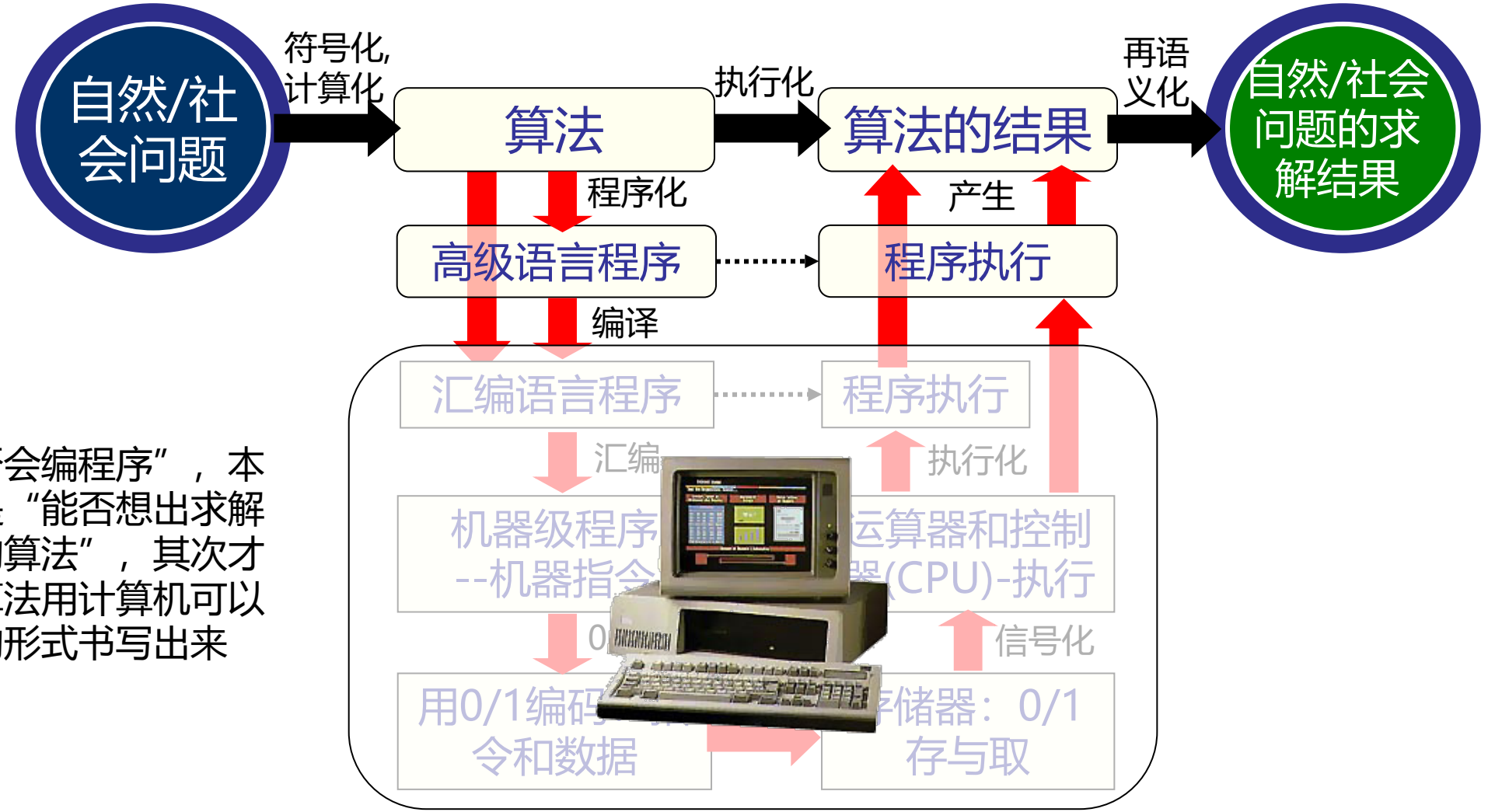
一、算法概述
二、问题与数学建模
三、算法策略
四、算法的表达-数据结构与控制结构
五、算法复杂性与计算复杂性

【示例】

TSP问题：问题与数学建模
TSP问题：算法策略
TSP问题：算法的表达
TSP问题：算法复杂性与计算复杂性

算法概述

算法的重要性
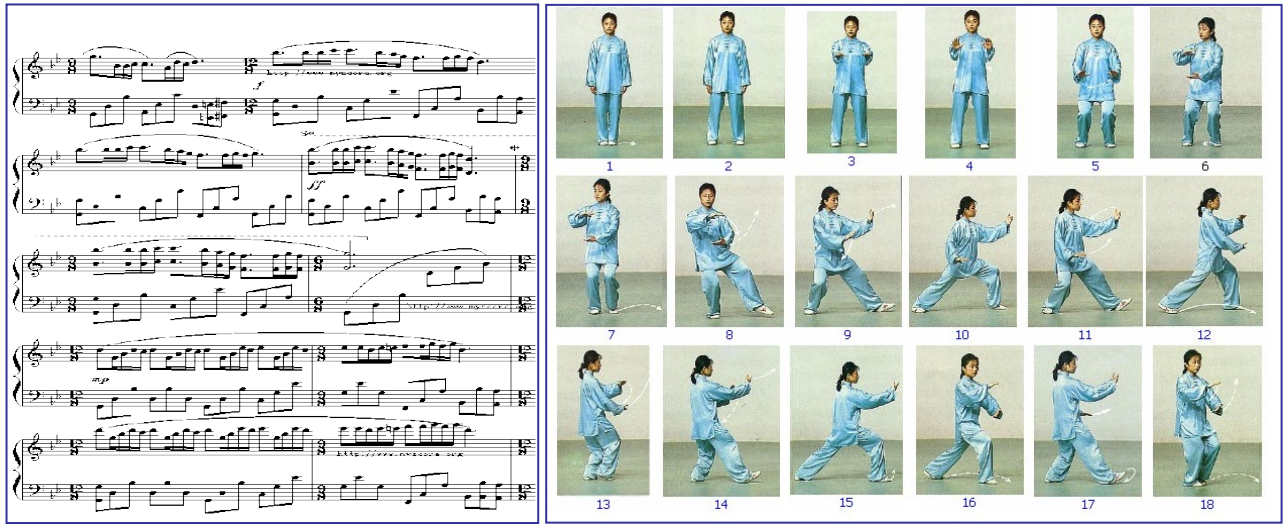


“是否会编程序”，本质上是“能否想出求解问题的算法”，其次才是将算法用计算机可以识别的形式书写出来

算法概述

什么是算法?

- 算法---计算机与软件的灵魂。“算法”(Algorithm)一词源于阿拉伯数学家阿科瓦里茨米(AlKhowarizmi)，其在公元825年写了著名的《波斯教科书》(Persian Textbook)，书中概括了进行四则算术运算的计算规则。
- 算法是一个有穷规则的集合，它用规则规定了解决某一特定类型问题的运算序列，或者规定了任务执行或问题求解的一系列步骤。



如音乐乐谱、太极拳谱等都可看作广义的算法

算法概述

算法示例

寻找两个正整数的最大公约数的欧几里德算法

输入：正整数*M*和正整数*N*

输出：*M*和*N*的最大公约数(设M>N)

算法步骤：

Step 1. *M*除以*N*,记余数为*R*

Step 2. 如果*R*不是*0*，将*N*的值赋给*M*, *R*的值赋给*N*, 返回Step 1; 否则, 最大公约数是*N*, 输出*N*, 算法结束。





	M	N	R	最大公约数
具体问题	32	24		?
算法计算过程				
1	32	24	8	
2	24	8	0	8
具体问题	31	11		?
算法计算过程				
1	31	11	9	
2	11	9	2	
3	9	2	1	
4	2	1	0	1

算法的表达

算法的模拟执行

算法概述

算法的五个特征

- **有穷性**：一个算法在执行有穷步规则之后必须结束。
- **确定性**：算法的每一个步骤必须要确切地定义，不得有歧义性。
- **输入**：算法有零个或多个的输入。
- **输出**：算法有一个或多个的输出/结果，即与输入有某个特定关系的量。
- **能行性**：算法中有待执行的运算和操作必须是相当基本的(可以由机器自动完成)，并能在有限时间内完成。

寻找两个正整数的最大公约数的欧几里德算法

输入：正整数*M*和正整数*N*

输出：*M*和*N*的最大公约数(设M>N)

算法步骤：

Step 1. *M*除以*N*,记余数为*R*

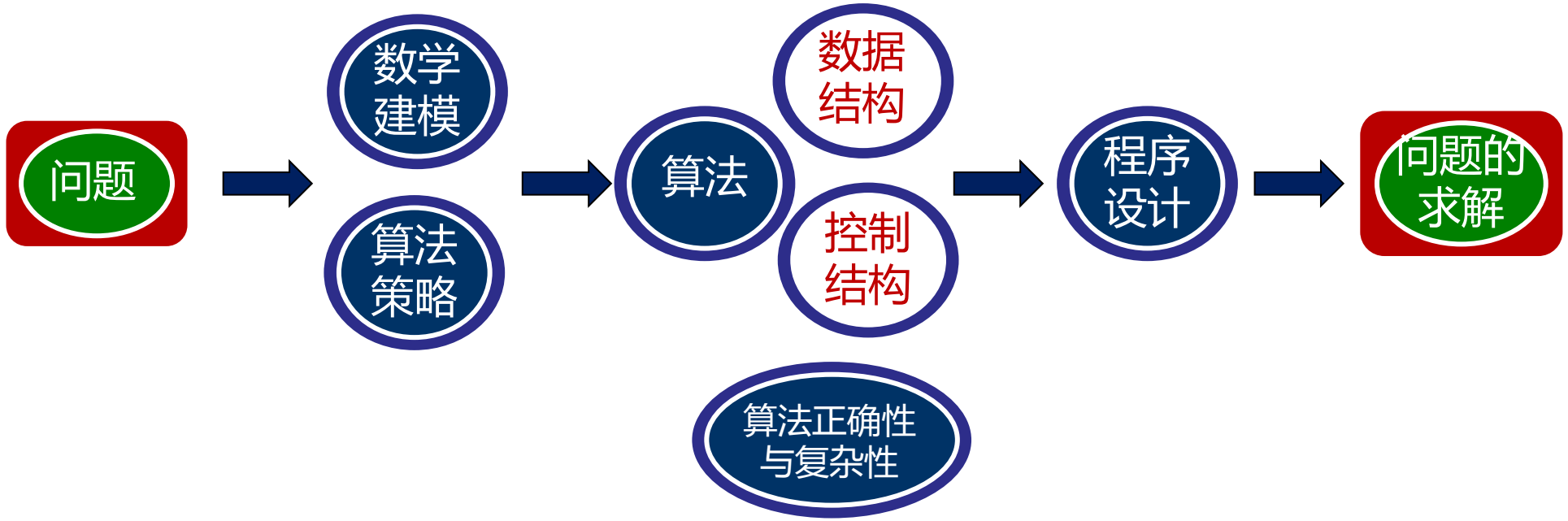
Step 2. 如果*R*不是*0*，将*N*的值赋给*M*, *R*的值赋给*N*, 返回Step 1; 否则, 最大公约数是*N*, 输出*N*, 算法结束。

基本运算：除法、赋值、逻辑判断

典型的“重复/循环”与“迭代”

算法概述

算法相关的知识



第7讲 算法：程序与计算系统之灵魂

一、算法概述

二、问题与数学建模

三、算法策略

四、算法的表达-数据结构与控制结构

五、算法复杂性与计算复杂性

【示例】

TSP问题：问题与数学建模

TSP问题：算法策略

TSP问题：算法的表达

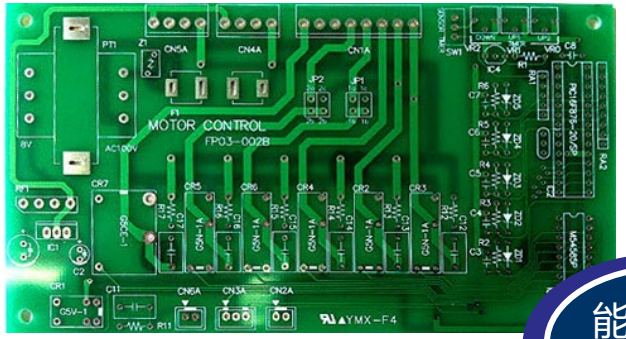
TSP问题：算法复杂性与计算复杂性

问题与数学建模

算法类问题

算法类问题：由一个算法可以解决的问题，寻找一个(唯一的)方法(算法)以解决同一类型的无穷多个单个问题系列的问题

- **问题组**
  - **【配送路径规划问题】**有n个地点需要送货，怎样一个次序，才能使送货距离最短
  - **【电路板机器钻孔路径规划】**有n个位置需要钻孔，怎样一个次序，才能使钻头移动距离最短
  - **【旅行商路径规划】**有n个城市需要旅行，怎样一个次序，才能使旅行者花费的交通成本最低



能发现什么吗?



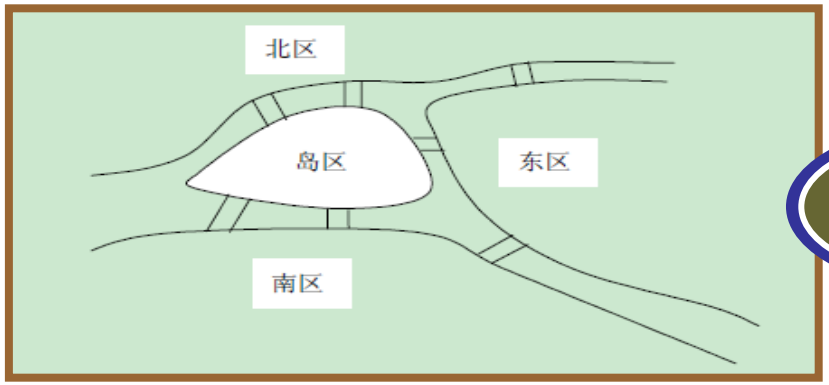


应用问题到算法类问题

问题与数学建模

算法类问题需要数学建模，而数学建模能发现问题的本质

【示例】哥尼斯堡七桥问题：“寻找走遍这7座桥且只许走过每座桥一次最后又回到原出发点的路径”。





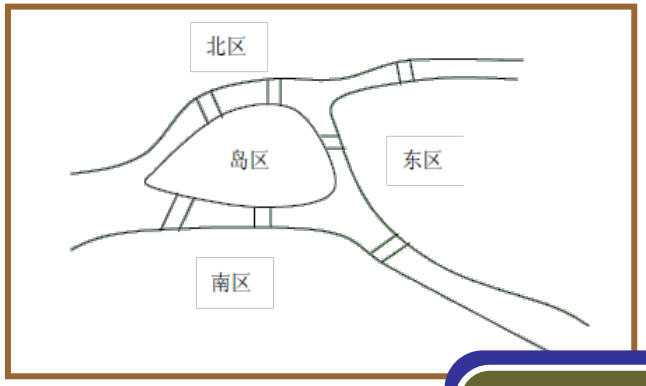
数学建模是用数学语言描述实际现象的过程，即建立数学模型的过程。

数学模型是对实际问题的一种数学表述，是关于部分现实世界为某种目的的一个抽象的简化的数学结构。

问题与数学建模

将社会/自然问题 转化为 数学问题

哥尼斯堡七桥问题： 抽象成一个 “图(Graph)” ,由节点和边所构成的一种结构。

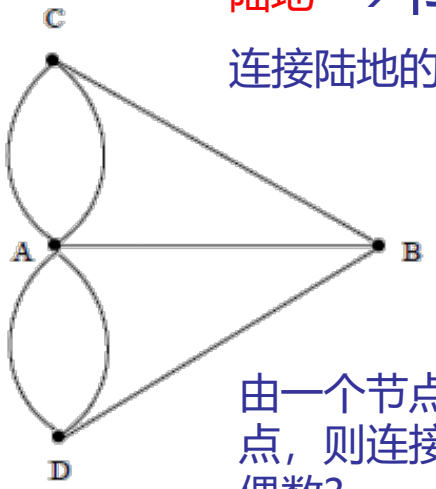


这个问题无解！
无解的问题还
用构造算法吗？



陆地--→节点

连接陆地的桥梁--→连接节点的边



由一个节点出发，又回到原出发节点，则连接该节点的边数是否应为偶数？

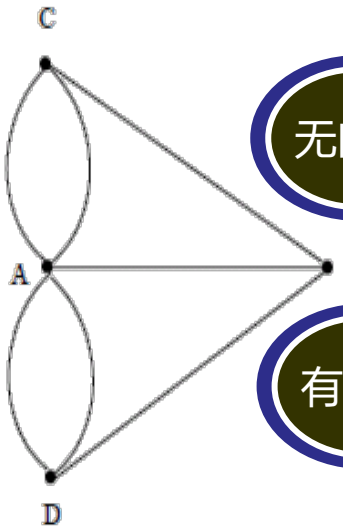
问题与数学建模

数学模型，可发现不同类别的问题及其性质

“连通----两个节点间有路径相连接”
“可达----从一个节点出发能够到达另一个节点”
“回路---从一个节点出发最后又回到该节点的一条路径”

经过图中每边一次且仅一次的回路

欧拉回路-欧拉图



无向图

节点的“度”

偶度节点

奇度节点

有向图

入度/出度

哈密尔顿回路-哈密尔顿图

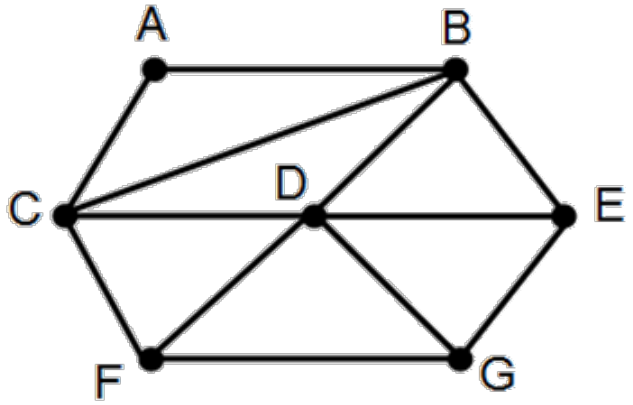
经过图中每节点一次且仅一次的回路

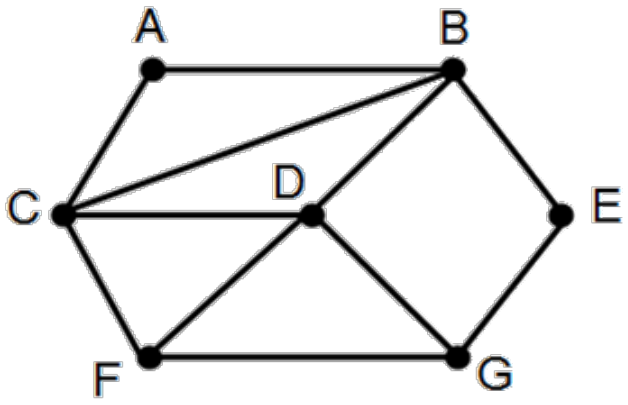
“图论”专门讲这方面的内容哟！



问题与数学建模

数学模型，可发现问题求解的基本思路





“一笔画”

- 凡是由偶度点组成的连通图，一定可以一笔画成。画时可以把任一偶度点为起点，最后一定能以这个点为终点画完此图；
- 凡是只有两个奇度点的连通图，其余都为偶度点，一定可以一笔画成。画时必须以一个奇度点为起点，另一个奇度点为终点。
- 其余情况都不能一笔画出。

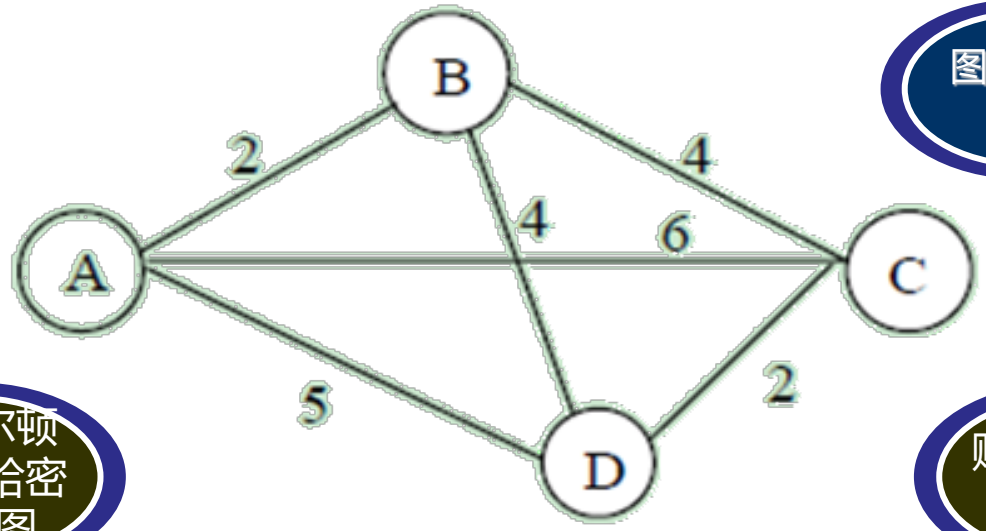


TSP问题：问题与数学建模

什么是TSP问题

◆TSP问题(Traveling Salesman Problem，旅行商问题)，威廉哈密尔顿爵士和英国数学家克克曼T.P.Kirkman于19世纪初提出TSP问题.

◆**TSP问题**：有若干个城市，任何两个城市之间的距离都是确定的，现要求一旅行商从某城市出发必须经过每一个城市且只能在每个城市逗留一次，最后回到原出发城市，问如何事先确定好一条最短的路线使其旅行的费用最少。



图来表达问题

经过图中每节点一次且仅一次的回路

哈密尔顿回路-哈密尔顿图

经过图中每节点一次且仅一次的回路；连接节点的边有“权值”

赋权-哈密尔顿图

TSP问题：问题与数学建模

将TSP问题抽象为数学问题

输入：n个城市，记为V={$v_1, v_2, \dots, v_n$}，任意两个城市$v_i, v_j \in V$之间有距离$d_{v_i v_j}$

输出：所有城市的一个访问顺序T={$t_1, t_2, \dots, t_n$}，其中$t_i \in V$，$t_{n+1} = t_1$，使得$min \sum_{i=1}^{n} d_{t_i t_{i+1}}$.

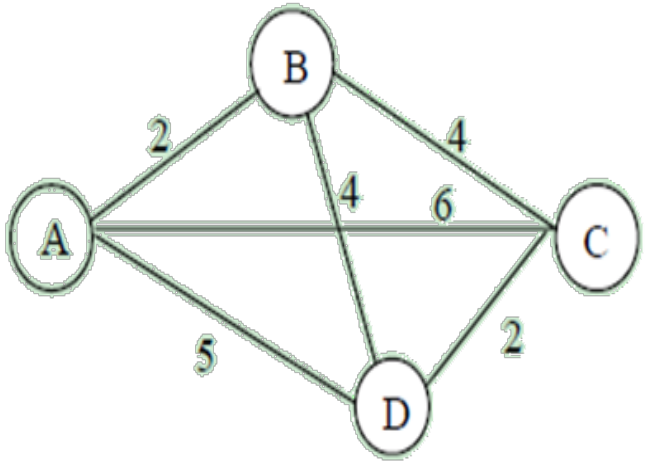
进一步简化

用整数编号每一个城市。用带下标的变量标记城市或城市间的特性

输入：n个城市，记为V={1, 2, …, n}，任意两个城市i，j之间的距离有$d_{i,j}$

输出：所有城市的一个访问顺序 T={$t_1, t_2, \dots, t_n$}，其中$t_i \in V$，$t_{n+1} = t_1$,使得$min \sum_{i=1}^{n} d_{t_i t_{i+1}}$.

问题求解的基本思想：在所有可能的访问顺序T构成的状态空间Ω上搜索使得$\sum_{i=1}^{n} d_{t_i t_{i+1}}$最小的访问顺序$T_{opt}$。



解的形式表示出来后，就可以遍历。

遍历：枚举-计算-验证，然后再优化

TSP问题：问题与数学建模

现实问题抽象后是否是相同的问题？

建模要素 \ 应用问题	配送路径规划问题	电路板机器钻孔路径规划	旅行商路径规划
问题表述	有n个地点需要送货，怎样一个次序，才能使送货距离最短	有n个位置需要钻孔，怎样一个次序，才能使钻头移动距离最短	有n个城市需要旅行，怎样一个次序，才能使旅行者花费的交通成本最低
输入/已知 【用整数及带下标变量表示特性】	n个地点可用编号1,...,n表示 任何两个地点i和j的距离表示为$d_{i,j}$	n个位置可用编号1,...,n表示 任何两个位置i和j的距离表示为$d_{i,j}$	n个城市可用编号1,...,n表示 任何两个城市i和j的交通成本表示为$d_{i,j}$
输出/结果 【可能解的形式】	求一路径T， $T=(T_1,T_2,...,T_n)$， 其中T_i为1...n中的某一个	求一路径T， $T=(T_1,T_2,...,T_n)$， 其中T_i为1...n中的某一个	求一路径T， $T=(T_1,T_2,...,T_n)$， 其中T_i为1...n中的某一个
约束 【满足约束的解是问题的解】	任意两个不同的i,j，则$T_i \neq T_j$	任意两个不同的i,j，则$T_i \neq T_j$	任意两个不同的i,j，则$T_i \neq T_j$
目标 【满足目标的解是问题的精确解】	T要使得 $\sum_{i=1}^{n} d_{T_iT_{i+1}}$ 最小 其中$d_{Tn,Tn+1}$为最后地点与出发地点的距离	T要使得 $\sum_{i=1}^{n} d_{T_iT_{i+1}}$ 最小 其中$d_{Tn,Tn+1}$为最后地点与出发地点的距离	T要使得 $\sum_{i=1}^{n} d_{T_iT_{i+1}}$ 最小 其中$d_{Tn,Tn+1}$为最后地点与出发地点的交通成本

TSP问题：问题与数学建模

现实问题抽象后是否是相同的问题？

建模要素 \ 应用问题	配送路径规划问题	电路板机器钻孔路径规划	旅行商路径规划
问题表述	有n个地点需要送货，怎样一个次序，才能使送货距离最短	有n个位置需要钻孔，怎样一个次序，才能使钻头移动距离最短	有n个城市需要旅行，怎样一个次序，才能使旅行者花费的交通成本最低
输入/已知 【用整数及带下标变量表示特性】	n个地点可用编号1,...,n表示 任何两个地点i和j的距离表示为$d_{i,j}$	n个位置可用编号1,...,n表示 任何两个位置i和j的距离表示为$d_{i,j}$	n个城市可用编号1,...,n表示 任何两个城市i和j的交通成本表示为$d_{i,j}$
输出/结果 【可能解的形式】	求一路径T， $T=(T_1,T_2,...,T_n)$， 其中T_i为1...n中的某一个	求一路径T， $T=(T_1,T_2,...,T_n)$， 其中T_i为1...n中的某一个	求一路径T， $T=(T_1,T_2,...,T_n)$， 其中T_i为1...n中的某一个
约束 【满足约束的解是问题的解】	任意两个不同的i,j，则$T_i \neq T_j$	任意两个不同的i,j，则$T_i \neq T_j$	任意两个不同的i,j，则$T_i \neq T_j$
目标 【满足目标的解是问题的精确解】	T要使得 $\sum_{i=1}^{n} d_{T_i T_{i+1}}$ 最小 其中$d_{Tn,Tn+1}$为最后地点与出发地点的距离	T要使得 $\sum_{i=1}^{n} d_{T_i T_{i+1}}$ 最小 其中$d_{Tn,Tn+1}$为最后地点与出发地点的距离	T要使得 $\sum_{i=1}^{n} d_{T_i T_{i+1}}$ 最小 其中$d_{Tn,Tn+1}$为最后地点与出发地点的交通成本

通过自然数及其下标将不同类型的数据关联起来，是**算法类问题抽象的重要方法**。表达四个核心要素：

- **输入**
- **输出**
- **约束**
- **目标**

第7讲 算法：程序与计算系统之灵魂

一、算法概述

二、问题与数学建模

三、算法策略

四、算法的表达-数据结构与控制结构

五、算法复杂性与计算复杂性

【示例】

TSP问题：问题与数学建模

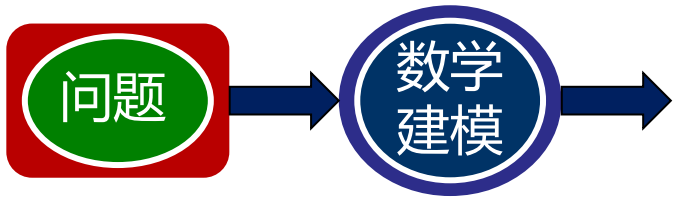
TSP问题：算法策略

TSP问题：算法的表达

TSP问题：算法复杂性与计算复杂性

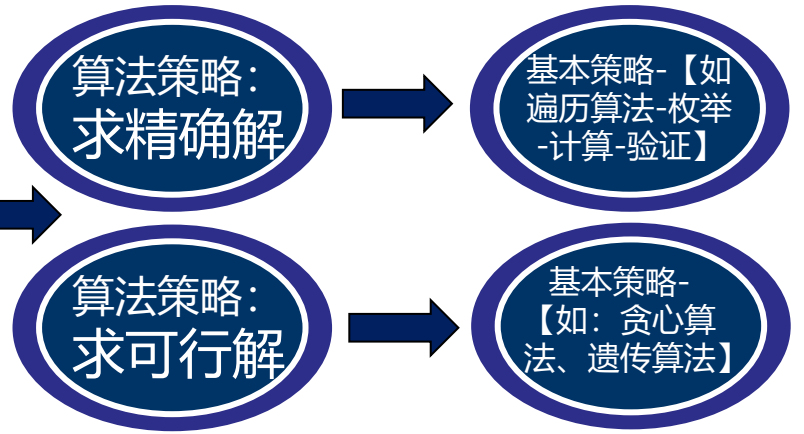
算法策略

算法策略是求解问题解的一种思想



自然数表征相关对象；带下标的变量将对象的不同特性数据关联起来：

- 输入【已知】
- 输出【**可能解的形式**】
- 约束【满足约束的解是问题的有效解—可行解】
- 目标【满足目标的解是问题的最优解—精确解】



【算法设计】在遍历策略基础上去掉无效计算量

【算法设计】以近似解换取计算量降低，有不同目的的近似解选择方法

算法策略

遍历策略基础上的算法设计示例

【示例】123456789给出一个5位数的排列来，使得前n位组成的整数能被n整除(n=1,2,3,4,5)。

$A*10^4+B*10^3+C*10^2+D*10^1+E*10^0$

(1) A%1=0; (2) $(A*10^1+B*10^0)$ %2=0; (3) $(A*10^2+B*10^1+C*10^0)$ %3=0

(4) $(A*10^3+B*10^2+C*10^1+D*10^0)$%4=0;; (5) $(A*10^4+B*10^3+C*10^2+D*10^1+E*10^0)$%5=0;

A到E 取其中之一{1，2，3，4，5，6，7，8，9}且A到E不重复

基本算法：枚举-计算-验证

- A-E, 任意组成一个排列：不重复
- 计算并依次检查条件(1)-(9)

算法策略

遍历策略基础上的算法设计示例

【示例】123456789给出一个5位数的排列来，使得前n位组成的整数能被n整除(n=1,2,3,4,5)。

$A*10^4+B*10^3+C*10^2+D*10^1+E*10^0$

(1) A%1=0; (2) $(A*10^1+B*10^0)$ %2=0; (3) $(A*10^2+B*10^1+C*10^0)$ %3=0

(4) $(A*10^3+B*10^2+C*10^1+D*10^0)$%4=0; (5) $(A*10^4+B*10^3+C*10^2+D*10^1+E*10^0)$%5=0;

A到E 取其中之一{ 1，2，

基本算法：枚举-计算-验证

- A-E, 任意组成一个排列：不重复
- 计算并依次检查条件(1)-(9)

```
For A=1 to 9 Step 1
{  For B=1 to 9 Step 1
   {  For C=1 to 9 Step 1
      {  For D=1 to 9 Step 1
         {  For E=1 to 9 Step 1
            {  if (  ((A*10+B) %2 ==0) &&
                     ((A*100+B*10+C) %3==0) &&
                     ((A*1000+B*100+C*10+D) %4==0) &&
                     ((A*10000+B*1000+C*100+D*10+E) %5==0) &&
                     A!=B && A!=C && A!=D && A!=E &&
                     B!=C && B!=D && B!=E &&
                     C!=D && C!=E && D!=E )
               { 输出"ABCDE"}
            }}}}}
```

单选题 1分

设置

22

测练题：右侧程序执行语句的次数，粗略估计是_____次。

A $9+9^2+9^3+9^4+3*9^5$

B 9^5

C 9^6

D n^5

```
For A=1 to 9 Step 1
{  For B=1 to 9 Step 1
   {  For C=1 to 9 Step 1
      {  For D=1 to 9 Step 1
         {  For E=1 to 9 Step 1
            {  if (  ((A*10+B) %2 ==0) &&
                     ((A*100+B*10+C) %3==0) &&
                     ((A*1000+B*100+C*10+D) %4==0) &&
                     ((A*10000+B*1000+C*100+D*10+E) %5==0) &&
                     A!=B && A!=C && A!=D && A!=E &&
                     B!=C && B!=D && B!=E &&
                     C!=D && C!=E && D!=E )
               { 输出"ABCDE"}
            }}}}}
```

提交

算法策略

遍历策略基础上的算法设计示例

【示例】123456789给出一个5位数的排列来，使得前n位组成的整数能被n整除(n=1,2,3,4,5)。

$A*10^4+B*10^3+C*10^2+D*10^1+E*10^0$

(1) A%1=0; (2) $(A*10^1+B*10^0)$ %2=0; (3) $(A*10^2+B*10^1+C*10^0)$ %3=0

(4) $(A*10^3+B*10^2+C*10^1+D*10^0)$%4=0;; (5) $(A*10^4+B*10^3+C*10^2+D*10^1+E*10^0)$%5=0;

A到E 取其中之一{1，2，3，4，5，6，7，8，9}且A到E不重复

基本算法：枚举-计算-验证

- A-E, 任意组成一个排列：不重复
- 计算并依次检查条件(1)-(9)

优化算法：减少无效计算量

哪些排列是没有意义的,是不需要计算和验证。(1) 前5位组成的数能被5整除，第5位只能是5。(2) 前2、4位组成的数能被2、4整除，则第2、4位只能是偶数，所以第1、3位只能是奇数。 A={1,3,5,7 }, B= {2,4,6,8}, C={1,3,5,7}, D= {2,4,6,8}, E=5。 此时，还要考虑A、C不能重复，B、D不能重复。

算法策略

遍历策略基础上的算法设计示例

【示例】123456789给出一个5位数的排列来，使得前n位组成的整数能被n整除(n=1,2,3,4,5)。

$A*10^4+B*10^3+C*10^2+D*10^1+E*10^0$

(1) A%1=0; (2) $(A*10^1+B*10^0)$

(4) $(A*10^3+B*10^2+C*10^1+D*10^0)\%5=0$

A到E 取其中之一{1，2，3，

基本算法：枚举-计算-验证

- A-E, 任意组成一个排列：不重复
- 计算并依次检查条件(1)-(9)

优化算

少无效

哪些排列是

，第5位只能

偶数，所以

D= {2,4,6,8

```
E = 5
For A=1 to 9 Step 2
{  For B=2 to 8 Step 2
   {  For C=1 to 9 Step 2
      {  if ( C  !=A )
         {   For D=2 to 8 Step 2
             {  if   (D!=B)
                {  if (  ((A*100+B*10+C) %3==0) &&
                    ((A*1000+B*100+C*10+D) %4==0) )
                   {输出"ABCDE"} }
             }
         }
   }}}
```

单选题 1分

设置

25

测练题：右侧程序执行语句的次数，粗略估计是_____次。

A $4^4*3+4^3+4^2+5$

B 4^5

C 4^6

D 4^4

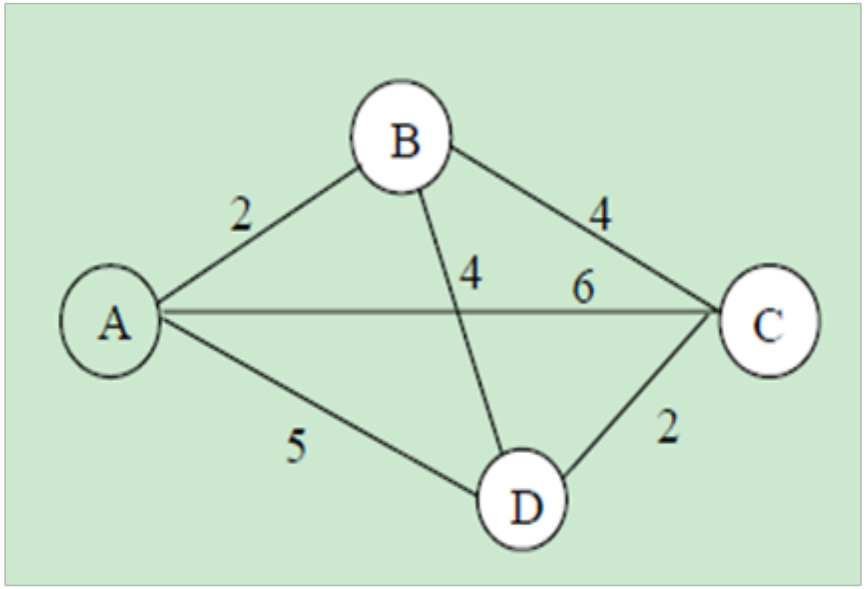
```
E = 5
For A=1 to 9 Step 2
{   For B=2 to 8 Step 2
   {  For C=1 to 9 Step 2
     {   if ( C  !=A )
        {    For D=2 to 8 Step 2
           {  if   (D!=B)
              {   if (  ((A*100+B*10+C) %3==0) &&
                   ((A*1000+B*100+C*10+D) %4==0) )
                  { 输出"ABCDE"} }
           }
        }
     }}}
```

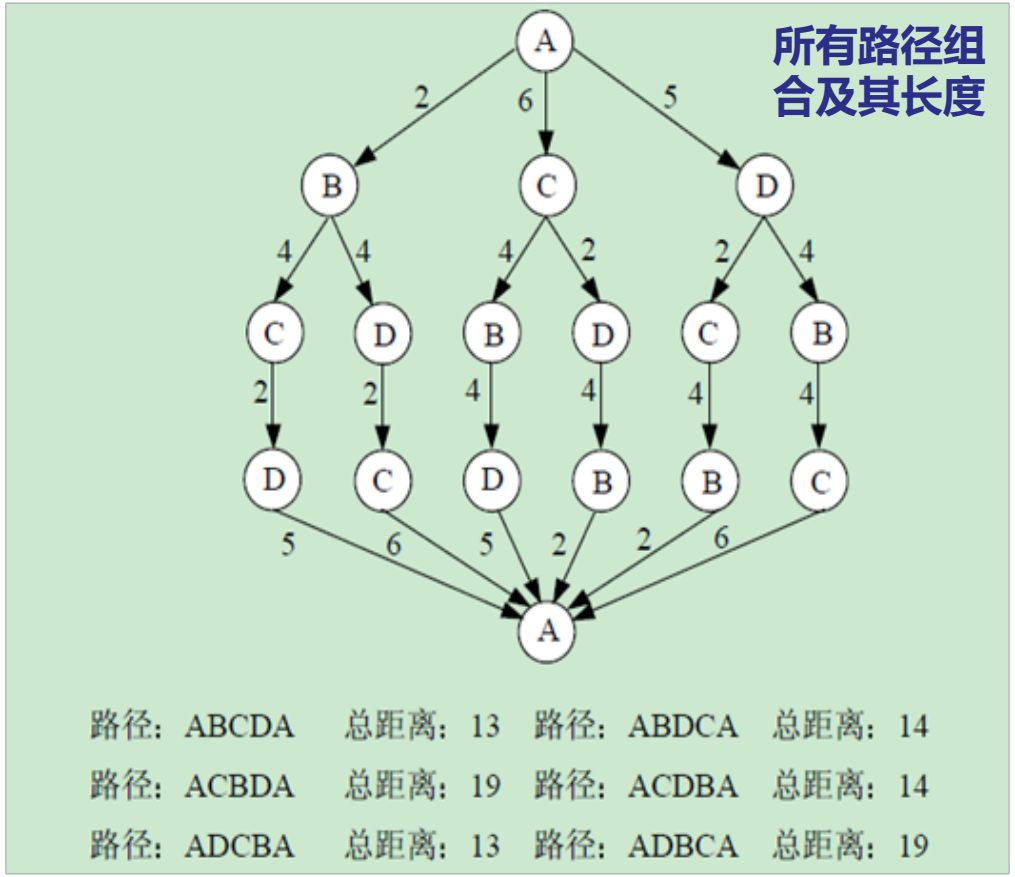
提交

TSP问题：算法策略

蛮干/遍历是基本的算法策略

遍历算法求解TSP问题：列出每一条可供选择的路线，计算出每条路线的总里程，最后从中选出一条最短的路线。



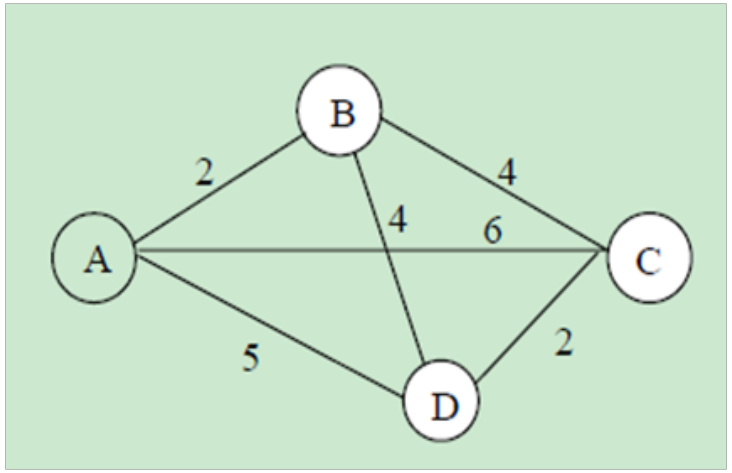


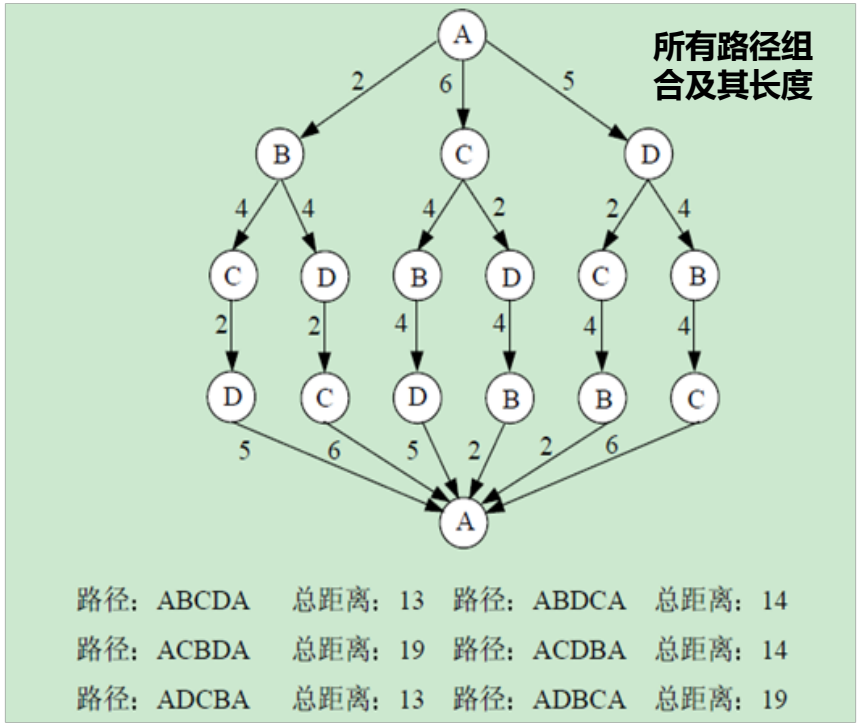
TSP问题：算法策略

蛮干/遍历策略会带来怎样的问题?

组合爆炸

- ✓路径组合数目：(n-1)!
- ✓20个城市，遍历总数1.216×10^{17}
- ✓计算机以每秒检索1000万条路线的计算速度，需386年。





TSP问题：算法策略

问题的难解性

◆TSP**问题的难解性**：随着城市数量**n**的上升，TSP问题的遍历计算量**(n-1)!** 剧增，计算资源将难以承受。

◆2001年解决了德国**15112**个城市的TSP问题，使用了美国Rice大学和普林斯顿大学之间互连的、速度为500MHz的Compaq EV6 Alpha 处理器组成的**110台计算机**，所有计算机花费的时间之和为**22.6年**。

An optimal TSP tour through Germany's 15 largest cities. It is the shortest among 43 589 145 600 possible tours visiting each city exactly once.



TSP问题：算法策略

寻找其他策略降低计算量

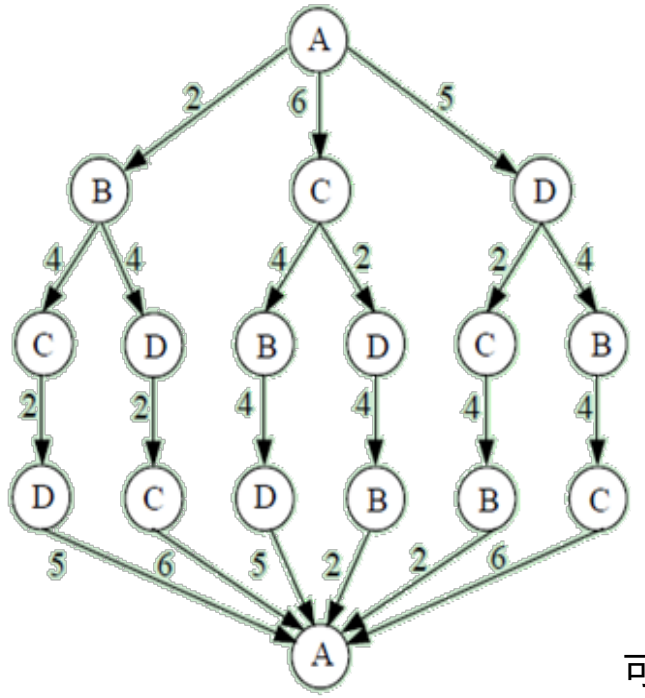
TSP问题，有没有其他求解算法呢？

◆最优解 vs. 可行解

◆不同的算法设计策略：

- ✓ 遍历、搜索算法
- ✓ 分治算法
- ✓ 贪心算法
- ✓ 动态规划算法
- ✓ ……

算法策略



可行解

最优解

路径：ABCDA 总距离：13　路径：ABDCA 总距离：14

路径：ACBDA 总距离：19　路径：ACDBA 总距离：14

路径：ADCBA 总距离：13　路径：ADBCA 总距离：19

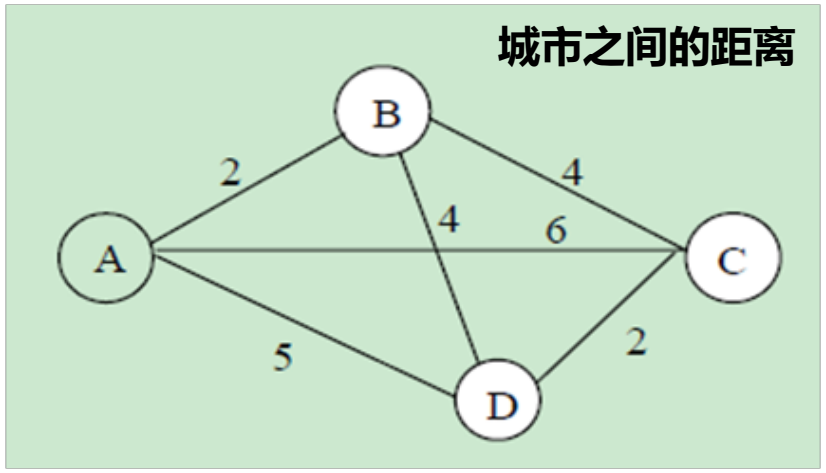
TSP问题的可行解与最优解示意

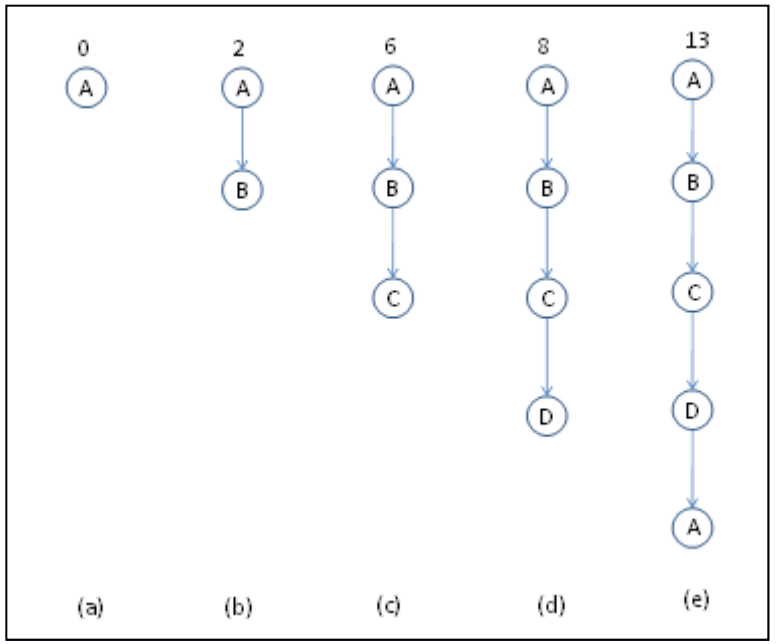
TSP问题：算法策略

另一种策略：贪心算法

贪心算法是一种算法策略。基本思想“今朝有酒今朝醉”

- ✓ 一定要做当前情况下的最好选择，否则将来可能会后悔，故名“贪心”。
- ✓ 从某一个城市开始，每次选择一个城市，直到所有城市都被走完。
- ✓ 每次在选择下一个城市的时候，只考虑当前情况，保证迄今为止经过的路径总距离最短。





吕志鹏教授团队代表性国际竞赛多次获奖





- **2021年国际计算机辅助设计会议（ICCAD）布局布线算法竞赛，全球第一**

EDA作为一切芯片设计活动的起点，也被誉为“芯片之母”。吕志鹏教授团队设计的启发式优化算法，在冗余导线检测、布线环路消除、并行化邻域评估加速、布局调整最优移动区域识别等多项关键技术上实现了突破。根据ICCAD 2021会议公布的竞赛结果，该团队设计的算法在所有测试算例上均达到了竞赛中的最优结果。据悉，这是吕志鹏教授团队首次参加该项大赛。本届CAD Contest算法竞赛共有来自12个国家/地区的137支队伍参与，包括众多国内外知名高校与研究机构，如加州伯克利分校、东京大学、台湾国立清华大学、台湾大学、香港中文大学、复旦大学等。

- **2022年DIMACS车辆路径优化算法挑战赛，两项全球第一**

中国研究生创“芯”大赛·EDA精英挑战赛我校获佳绩

- 2024中国研究生创“芯”大赛·EDA精英挑战赛（简称“EDA精英挑战赛”）落幕。计算机学院吕志鹏教授团队获全国一等奖1项。团队由吕志鹏、苏宙行共同指导，参赛学生为博士生罗灿辉。团队针对“FPGA 面向布线优化的详细布局”赛道的挑战难题，提出高效的启发式优化算法，实现了面向FPGA布线优化的智能详细布局优化引擎。该团队还获得了安路科技企业特别奖。

我院师生在2025第七届集成电路EDA精英挑战赛总决赛中斩获佳绩

- 我院一等奖获奖团队来自冯丹教授和刘康副研究员课题组，参赛选手为硕士研究生王玉佳、张昭骏和孙亦康。该团队针对“超大规模版图模板分类算法”赛道的挑战难题，面向芯片版图聚类中的余弦相似度和多边形边偏移等约束场景提出了高效的版图聚类算法，聚类耗时相比于大赛提供的基线算法降低100倍以上。
- 我院另一支一等奖获奖团队来自吕志鹏教授和苏宙行副研究员课题组，参赛选手为博士研究生罗灿辉。该团队针对“支持重新组网的多FPGA系统布线”赛道的挑战难题，提出了高效的启发式优化算法，实现多FPGA系统的智能布线引擎，基于负载均衡组网以及候选路径布线技术，在多个系统案例中优化得到最低通信时延，并获得思尔芯企业特别奖。



证书编号：2825A10012

扫码验真

CPIPC 中国研究生创新实践系列大赛

获奖证书

2025 中国研究生创“芯”大赛 · EDA 精英挑战赛

一等奖

团队成员：王玉佳、张昭骏、孙亦康

指导教师：刘康、冯丹

中国学位与研究生教育学会　　中国科协青少年科技中心

二〇二五年十一月



证书编号：2825A10005

扫码验真

CPIPC 中国研究生创新实践系列大赛

获奖证书

2025 中国研究生创“芯”大赛 · EDA 精英挑战赛

一等奖

团队成员：罗灿辉

指导教师：吕志鹏、苏宙行

中国学位与研究生教育学会　　中国科协青少年科技中心

二〇二五年十一月

第7讲 算法：程序与计算系统之灵魂

一、算法概述

二、问题与数学建模

三、算法策略

四、算法的表达-数据结构与控制结构

五、算法复杂性与计算复杂性

【示例】

TSP问题：问题与数学建模

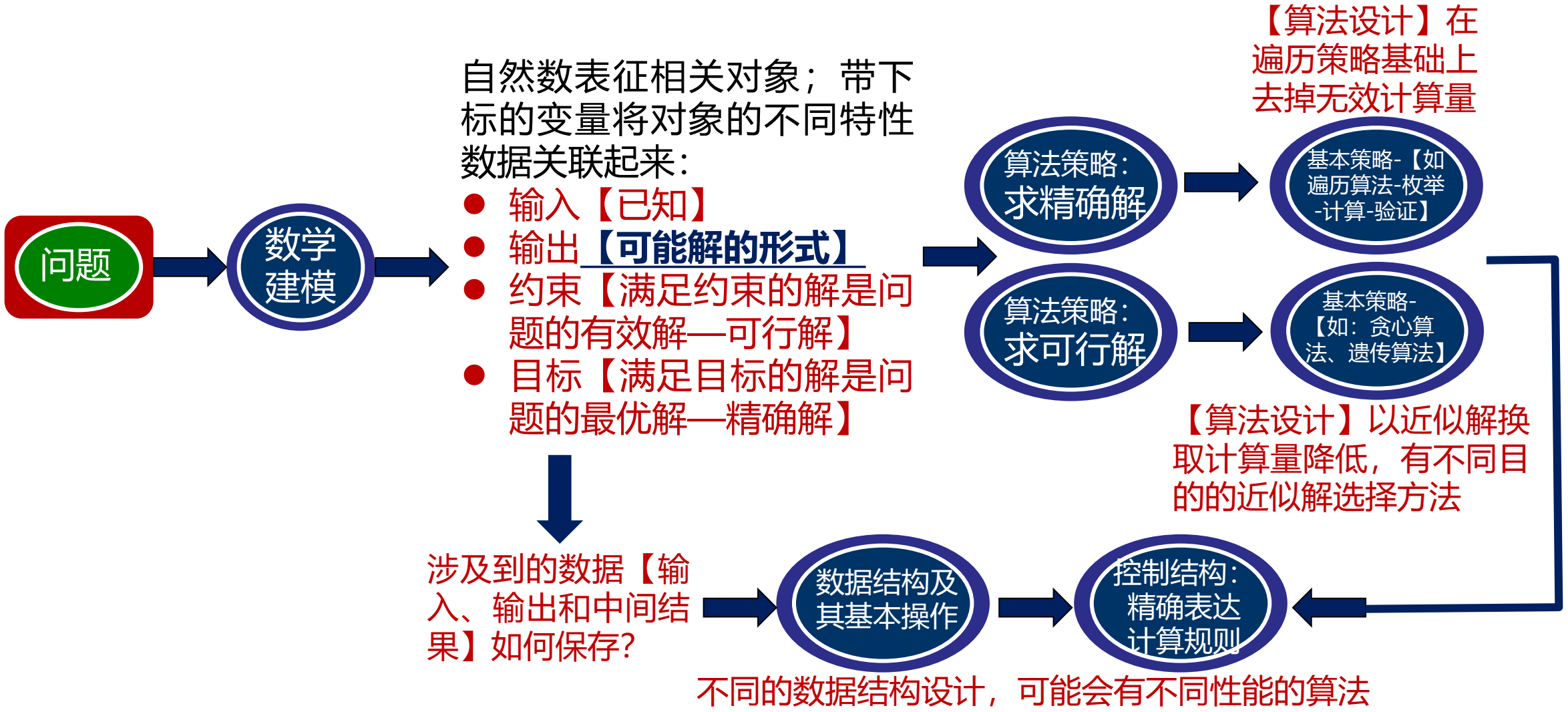
TSP问题：算法策略

TSP问题：算法的表达

TSP问题：算法复杂性与计算复杂性

算法的表达-数据结构与控制结构

算法的表达需要表达数据及计算规则



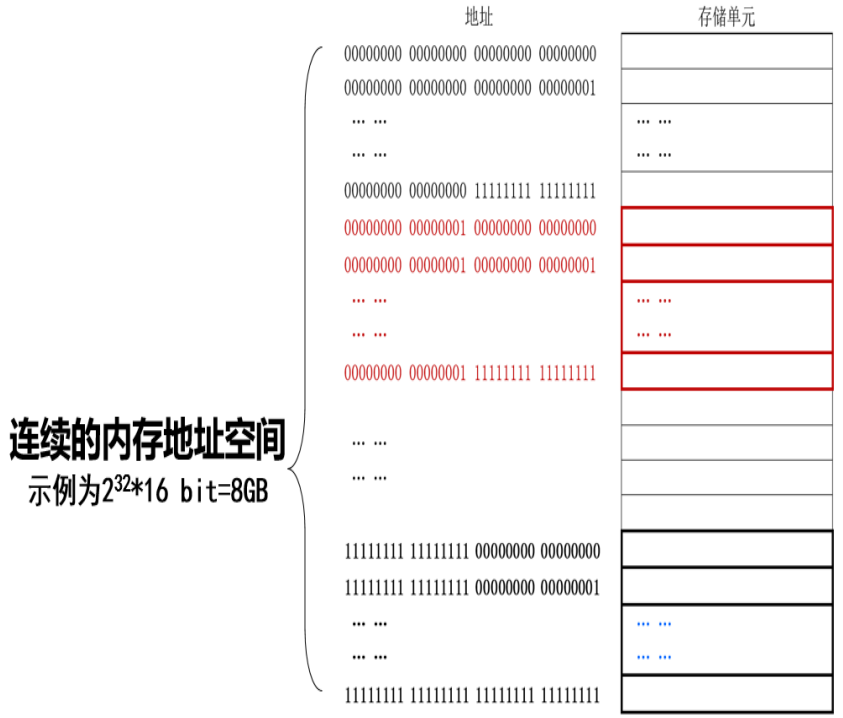
算法的表达-数据结构与控制结构

为什么需要数据结构

数据结构是数据的逻辑结构、存储结构及其操作的总称，它提供了问题求解/算法的数据操纵机制。

- 机器：CPU能够处理的数据需要保存在存储器。【存储单元】 vs. 【地址】
- 算法：【变量】vs. 【变量名】、【变量类型】





算法的表达-数据结构与控制结构

变量 vs. 存储单元

高级语言

- 用变量保存数据。变量名 vs. 变量值
- 变量使用前要声明变量的类型。例如整型int、长整型long int、字符型char等。
- 变量在程序中可通过赋值改变其变量值

```
int X = 23;
char Y =  'AB' ;
Char Z =  'ABCD' ;
```

机器

- 所有数据都需要保存在存储器中
- 不同变量类型（1）占用不同个数的存储单元；（2）存储单元中数的表示方法可能是不同的。
- 一般整型数int占用2个存储单元，用二进制补码表示数值，而长整型long int 占用4个存储单元
- 字符型char，是一个字母占用一个存储单元，用ASCII码表示数值
- 变量在被装入内存时，建立了变量名与存储单元地址的映射关系



	存储地址	存储内容		
X	00000000 00000001	00000000	00010111	二进制的23
Y	00000000 00000010	01000001	01000010	ASCII码AB
Z	00000000 00000011	01000001	01000010	ASCII码AB
	00000000 00000100	01000011	01000100	ASCII码CD
	00000000 00000101	00000000		
	00000000 00000110			

算法的表达-数据结构与控制结构

其他数据结构：指针变量 与 存储单元

◆指针是一种内存地址型变量，即其存储的是个内存地址，那个地址对应的存储单元才是指针型变量实际的值。

String *P=”ABCD”;	P
String v = “ABCD”;	v *p

存储地址	存储内容	
00000000 00000000	00000100	00001000
00000000 00000001	00000000	00000100
00000000 00000010	00001100	00001010
00000000 00000011	00000000	00000000
00000000 00000100	01000001	01000010
00000000 00000101	01000011	01000100
00000000 00000110	00000000	00000000
00000000 00000111	01000001	01000010
00000000 00001000	01000011	01000100
00000000 00001001	00000000	00000000

算法的数据结构及其操作

不同的变量类型 与 存储单元

用名字表示的存储地址，即变量名	存储地址	存储内容(即变量值)
Mark	00000000 00000000 00000000 00000001 00000000 00000010 00000000 00000011	(注：可通过赋值发生改变)
Sum	00000000 00000100 00000000 00000101	(注：可通过赋值发生改变)
Distance	00000000 00000110 00000000 00000111	(注：可通过赋值发生改变)

算法的数据结构及其操作

多元素变量 与 存储单元

◆**向量**或**列表**是有序数据的集合型变量，向量中的每一个元素都属于同一个数据类型，用一个统一的向量名和下标来唯一的确定向量中的元素。在程序设计语言中，又称为**数组**。

◆向量名通常表示该向量的起始存储地址，而向量下标表示所指向元素相对于起始存储地址的偏移位置。

向量实例

82	Mark[0]
95	Mark[1]
100	Mark[2]
60	Mark[3]
80	Mark[4]

编写求上述数组中值的平均值的程序

```
n = 4;
Sum=0;
For  J =0 to n Step 1
{    Sum = Sum + mark[ J ];
        }
Next  J
Avg = Sum/(n+1);
```

用变量名和元素位置共同表示存储地址，即向量		存储地址	存储内容(即变量值)
Mark	[0]	00000000 00000000 00000000 00000001	(注：82 的 4 字节二进制数可通过赋值发生改变)
	[1]	00000000 00000010 00000000 00000011	(注：95 的 4 字节二进制数可通过赋值发生改变)
	[2]	00000000 00000100 00000000 00000101	(注：100 的 4 字节二进制数可通过赋值发生改变)
	[3]	00000000 00000110 00000000 00000111	(注：60 的 4 字节二进制数可通过赋值发生改变)
	[4]	00000000 00001000 00000000 00001001	(注：80 的 4 字节二进制数可通过赋值发生改变)

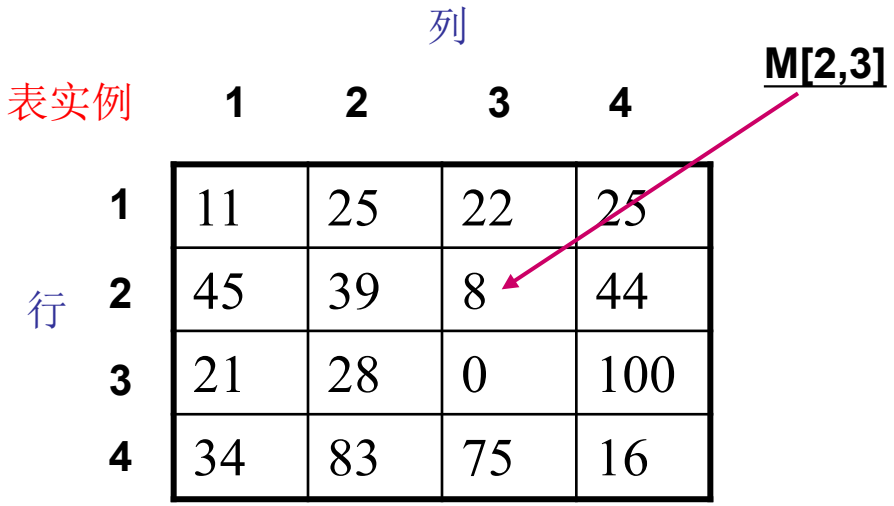
向量存储实例

多元素变量使得程序可通过下标来操作变量中的每一个元素

算法的数据结构及其操作

多元素变量 与 存储单元

◆**矩阵**或**表**是按行按列组织数据的集合型变量，通常是一个二维向量，可表示为如M[2,3]或M[2][3]形式，即用符号名加两个下标来唯一确定表中的一个元素，前一下标为行序号，后一下标为列序号。系统会自动将其转换为对应的存储地址，找到相应的存储单元。在程序设计语言中，矩阵或表是一个多维数组变量。



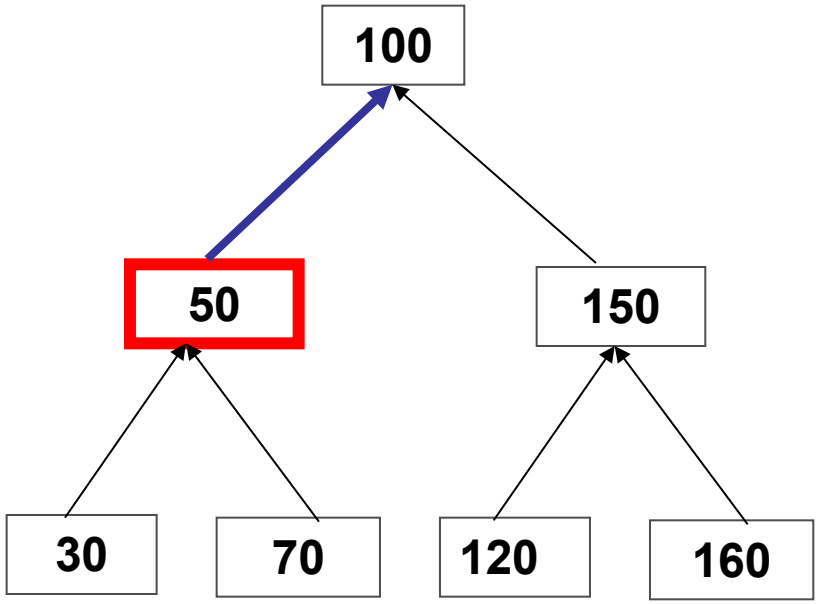

```
Sum=0;
For  I =1 to 4 Step 1
{   For  J =1 to 4 Step 1
    { Sum = Sum + M[I][J];  }
    Next  J
}
Next  I
Avg = Sum/16;
```

逻辑上是二维的按行、列下标来操作一个元素，如M[2,3]或M[2][3]；物理上仍旧是一维存储的，由“表起始地址+ (行下标-1)*列数+(列下标-1)”。这种转换可由系统自动完成，程序中只需按下标操作即可，即如M[2][3]

算法的数据结构及其操作

典型的数据结构-- “树” ：一种存储方法

“树”的**存储结构**: 一个存储单元存储 “数据元素”；一个存储单元存储“指针”，指示数据元素之间的逻辑关系



数据的逻辑结构

数据的存储结构

数据元素

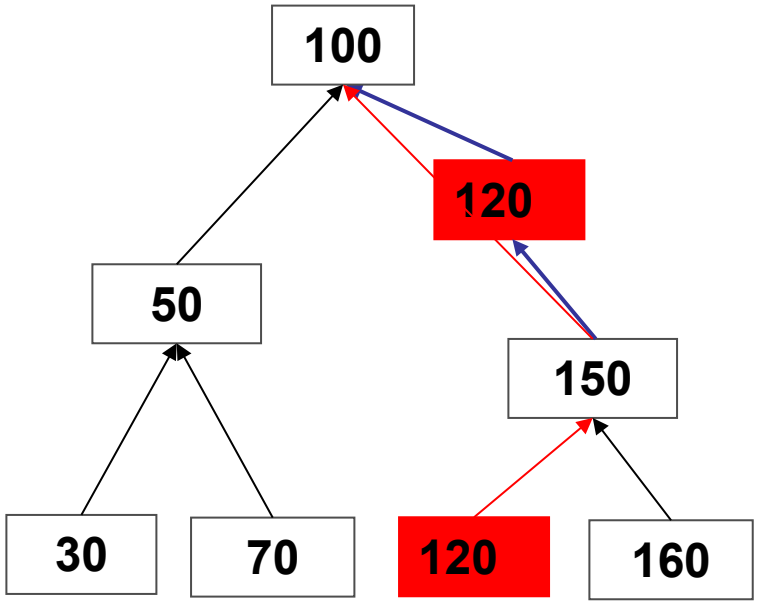
对应数据元素的指针—指向该元素的父元素

存储地址	存储内容(即变量值)	注释
00000000 00000001	00000000 01100100	第 1 个数据元素 100
00000000 00000010	00000000 00000000	第 1 个数据元素的指针
00000000 00000011	00000000 00110010	第 2 个数据元素 50
00000000 00000100	00000000 00000001	第 2 个数据元素的指针
00000000 00000101	00000000 10100110	第 3 个数据元素 150
00000000 00000110	00000000 00000001	第 3 个数据元素的指针
00000000 00000111	00000000 00011110	第 4 个数据元素 30
00000000 00001000	00000000 00000011	第 4 个数据元素的指针
00000000 00001001	00000000 01000110	第 5 个数据元素 70
00000000 00001010	00000000 00000011	第 5 个数据元素的指针
00000000 00001011	00000000 01111000	第 6 个数据元素 120
00000000 00001100	00000000 00000101	第 6 个数据元素的指针
00000000 00001101	00000000 10100000	第 7 个数据元素 160
00000000 00001110	00000000 00000101	第 7 个数据元素的指针
00000000 00001111		

算法的数据结构及其操作

典型的数据结构-- “树”：一种存储方法

存储结构中，通过指针的变化，不改变数据元素的存储，但却改变了数据元素之间的逻辑关系



数据的逻辑结构

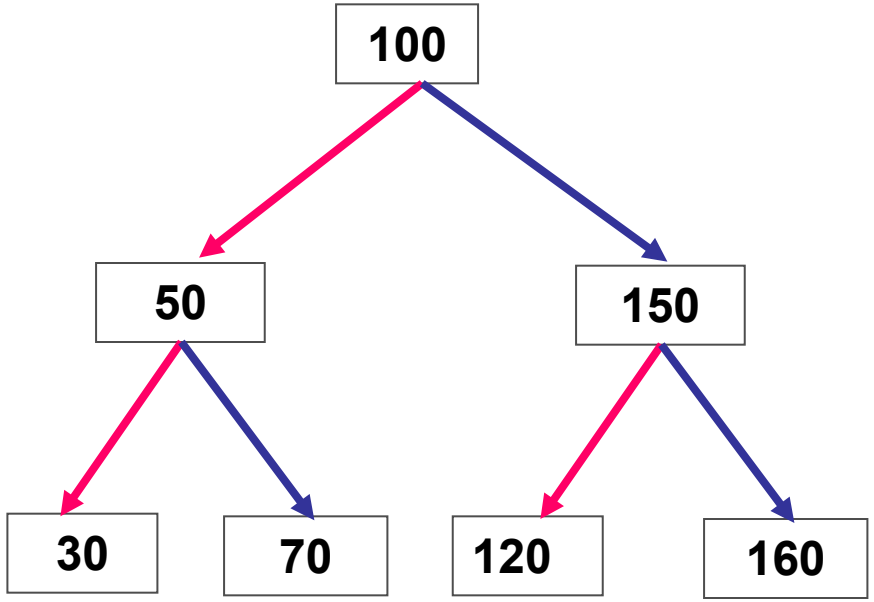
数据的存储结构

存储地址	存储内容(即变量值)	注释
00000000 00000001	00000000 01100100	第 1 个数据元素 100
00000000 00000010	00000000 00000000	第 1 个数据元素的指针
00000000 00000011	00000000 00110010	第 2 个数据元素 50
00000000 00000100	00000000 00000001	第 2 个数据元素的指针
00000000 00000101	00000000 10100110	第 3 个数据元素 150
00000000 00000110	00000000 00001011	第 3 个数据元素的指针
00000000 00000111	00000000 00011110	第 4 个数据元素 30
00000000 00001000	00000000 00000011	第 4 个数据元素的指针
00000000 00001001	00000000 01000110	第 5 个数据元素 70
00000000 00001010	00000000 00000011	第 5 个数据元素的指针
00000000 00001011	00000000 01111000	第 6 个数据元素 120
00000000 00001100	00000000 00000001	第 6 个数据元素的指针
00000000 00001101	00000000 10100000	第 7 个数据元素 160
00000000 00001110	00000000 00000101	第 7 个数据元素的指针
00000000 00001111		

算法的数据结构及其操作

典型的数据结构-- “树”：另一种存储方法

“树”的另一种**存储结构**，用两个指针表达数据之间的逻辑关系，一个指向其左数据元素，一个指向其右数据元素。



数据的逻辑结构

数据的存储结构

数据元素

对应数据元素的左指针—指向该元素的左侧子结点

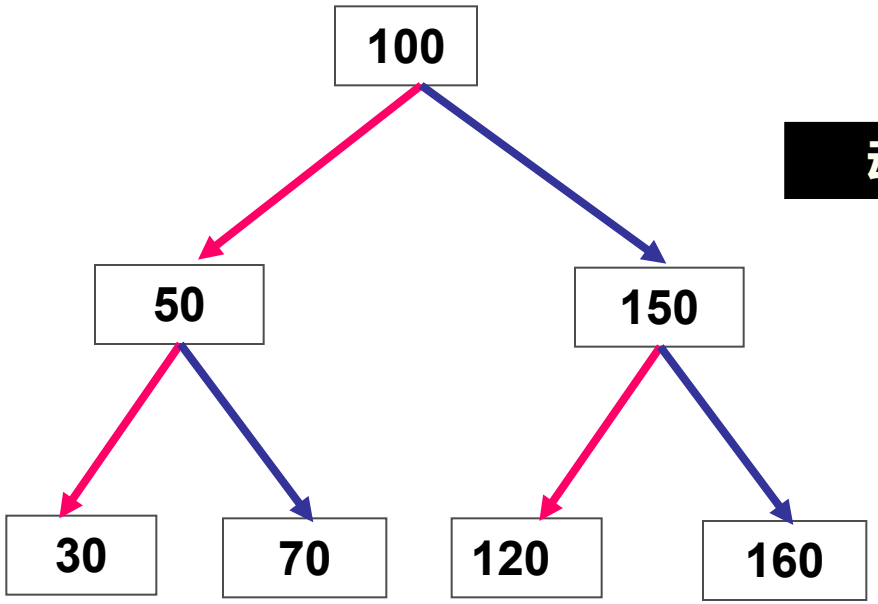
对应数据元素的右指针—指向该元素的右侧子结点

	存储地址	存储内容(即变量值)	注释
TreeElement	00000000 00000001	00000000 01100100	第 1 个数据元素 100
	00000000 00000010	00000000 00110010	第 2 个数据元素 50
	00000000 00000011	00000000 10100110	第 3 个数据元素 150
	00000000 00000100	00000000 00011110	第 4 个数据元素 30
	00000000 00000101	00000000 01000110	第 5 个数据元素 70
	00000000 00000110	00000000 01111000	第 6 个数据元素 120
	00000000 00000111	00000000 10100000	第 7 个数据元素 160
	00000000 00001000		
LeftPointer	00000000 00001001	00000000 00000010	第 1 个数据元素的左指针
	00000000 00001010	00000000 00000100	第 2 个数据元素的左指针
	00000000 00001011	00000000 00000110	第 3 个数据元素的左指针
	00000000 00001100	00000000 00000000	第 4 个数据元素的左指针
	00000000 00001101	00000000 00000000	第 5 个数据元素的左指针
	00000000 00001110	00000000 00000000	第 6 个数据元素的左指针
	00000000 00001111	00000000 00000000	第 7 个数据元素的左指针
RightPointer	00000000 00010000	00000000 00000011	第 1 个数据元素的右指针
	00000000 00010001	00000000 00000101	第 2 个数据元素的右指针
	00000000 00010010	00000000 00000111	第 3 个数据元素的右指针
	00000000 00010011	00000000 00000000	第 4 个数据元素的右指针
	00000000 00010100	00000000 00000000	第 5 个数据元素的右指针
	00000000 00010101	00000000 00000000	第 6 个数据元素的右指针
	00000000 00010110	00000000 00000000	第 7 个数据元素的右指针

数据结构不同，数据之间的操作方法也是不同的

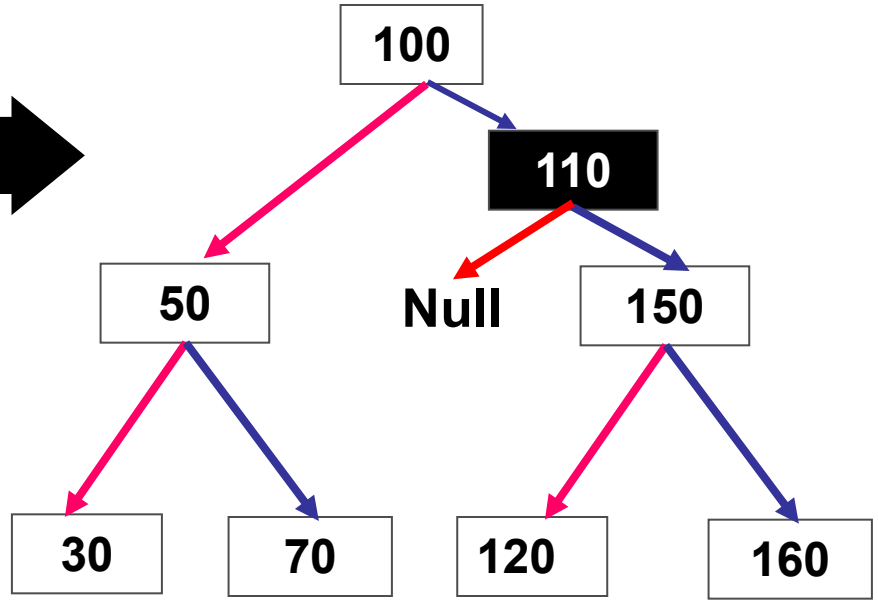
算法的数据结构及其操作

练习一下...



数据的逻辑结构

动手练习一下



数据的逻辑结构

TSP问题：算法的表达

问题求解相关数据如何存储?

城市映射为编号: A---1, B---2, C---3, D---4

城市间距离关系: 表或二维数组D，用D[i][j]或D[i,j]来确定欲处理的每一个元素

城市编号	1	2	3	4
1		2	6	5
2	2		4	4
3	6	4		2
4	5	4	2	

D

D[2][3]

访问路径/解: 一维数组S，用S[j]来确定每一个元素

S	1	4	3	2
	S[1]	S[2]	S[3]	S[4]

{A->D->C->B->A}

算法的表达-数据结构与控制结构

算法的类自然语言表达方法

算法与程序的基本控制结构

✓顺序结构：“执行A，然后执行B”，是按顺序执行一条条规则的一种结构 。

✓分支结构：“如果Q成立，那么执行A，否则执行B”，Q是某些逻辑条件，即按条件判断结果决定执行哪些规则的一种结构 。

✓循环结构：控制指令或规则的多次执行的一种结构---迭代(iteration)

◆循环结构又分为有界循环结构和条件循环结构。

✓有界循环：“执行A指令N次”，其中N是一个整数。

✓条件循环：某些时候称为无界循环，“重复执行A直到条件Q不成立”或“当Q成立时反复执行A”，其中Q是条件。

算法的表达-数据结构与控制结构

示例

【示例】sum=1+2+3+4+...+n求和问题的算法描述

Start of the algorithm(算法开始)

(1)输入N的值;

(2)设 i 的值为1； sum的值为0;

(3)如果 i<=N，则执行第(4)步，否则转到第(7)步执行;

(4)计算 sum + i，并将结果赋给sum;

(5)计算 i+1，并将结果赋给i;

(6)返回到第3步继续执行;

(7)输出sum的结果。

End of the algorithm(算法结束)

自然语言表示的算法容易出现二义性、不确定性等问题

算法的表达-数据结构与控制结构

算法的程序流程图表达方法

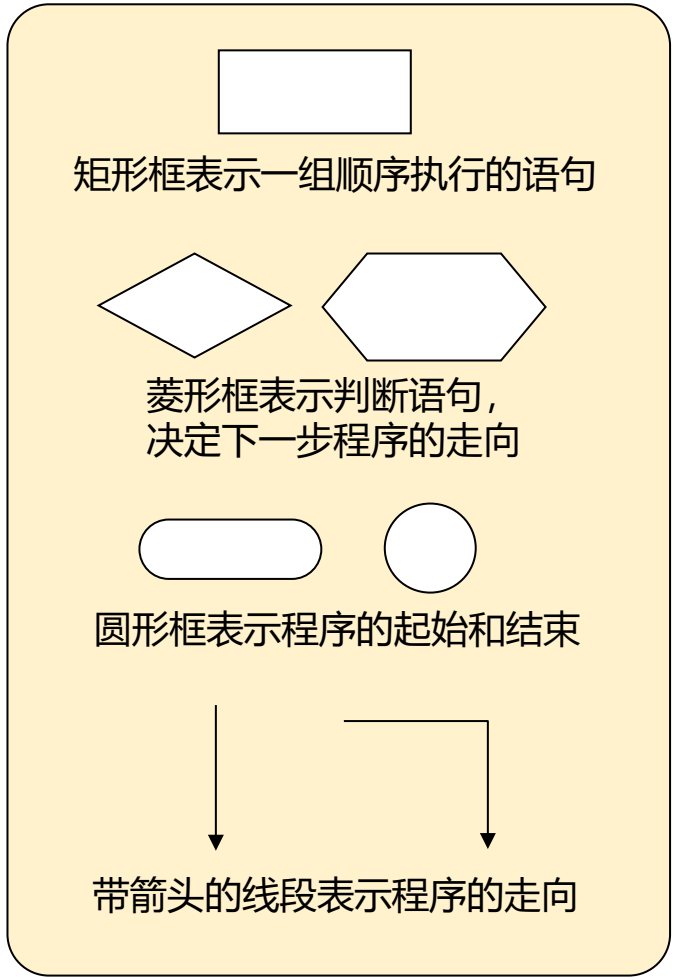
流程图的基本表示符号

✓矩形框：表示一组顺序执行的规则或者程序语句。

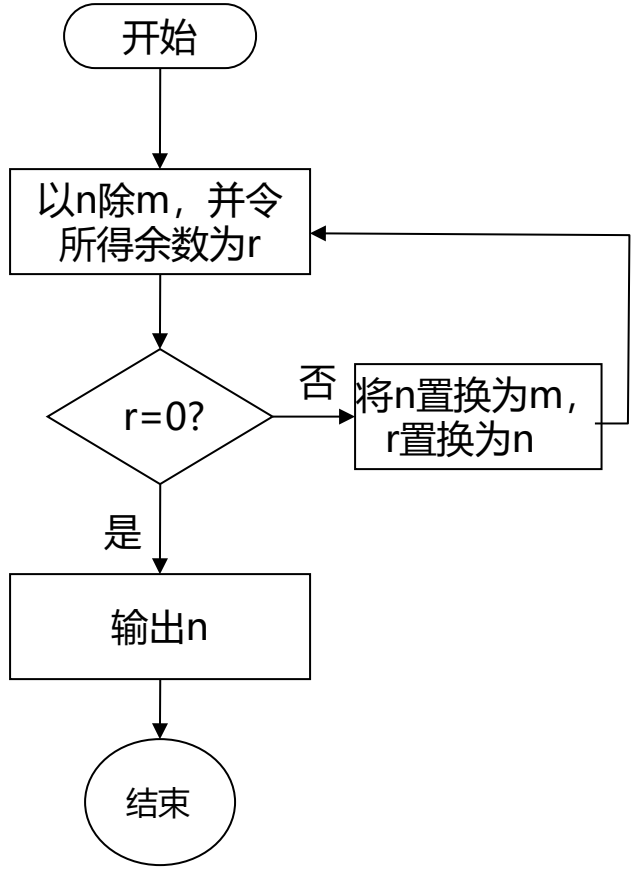
✓菱形框：表示条件判断，并根据判断结果执行不同的分支。

✓圆形框：表示算法或程序的开始或结束。

✓箭头线：表示算法或程序的走向。



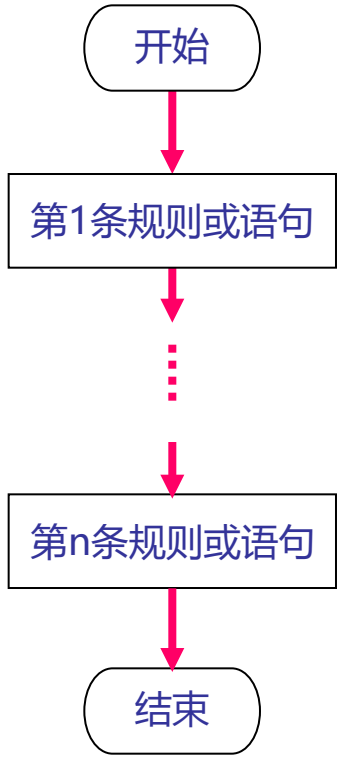
【示例】欧几里德算法流程图



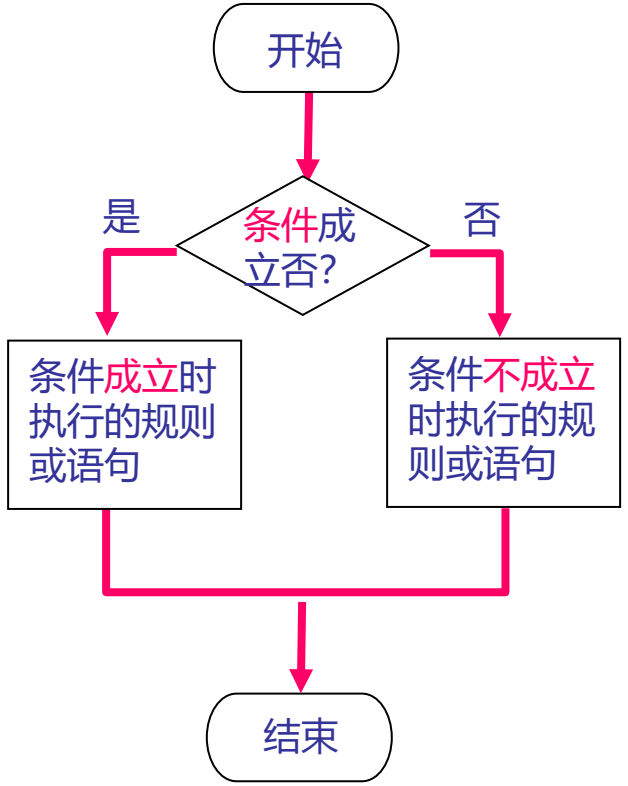
算法的表达-数据结构与控制结构

算法的程序流程图表达方法

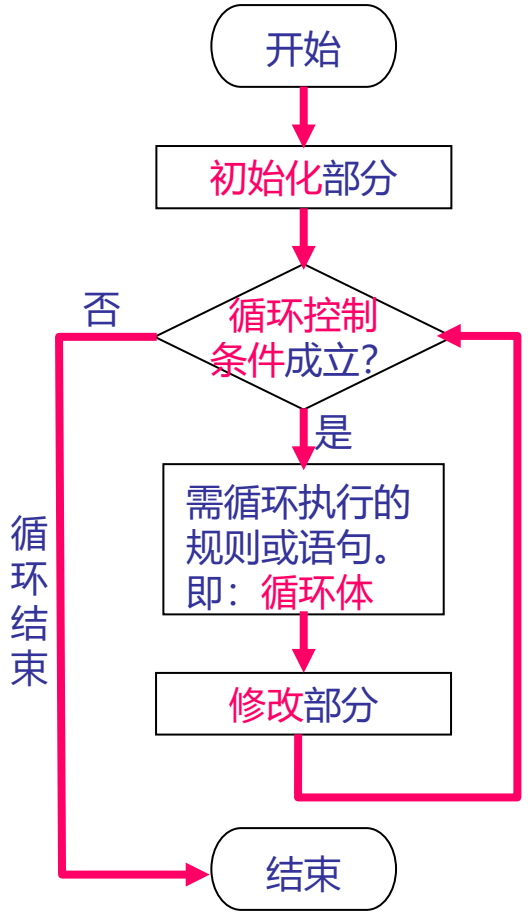
三种控制结构的流程图表达



顺序结构的流程图



分支结构的流程图



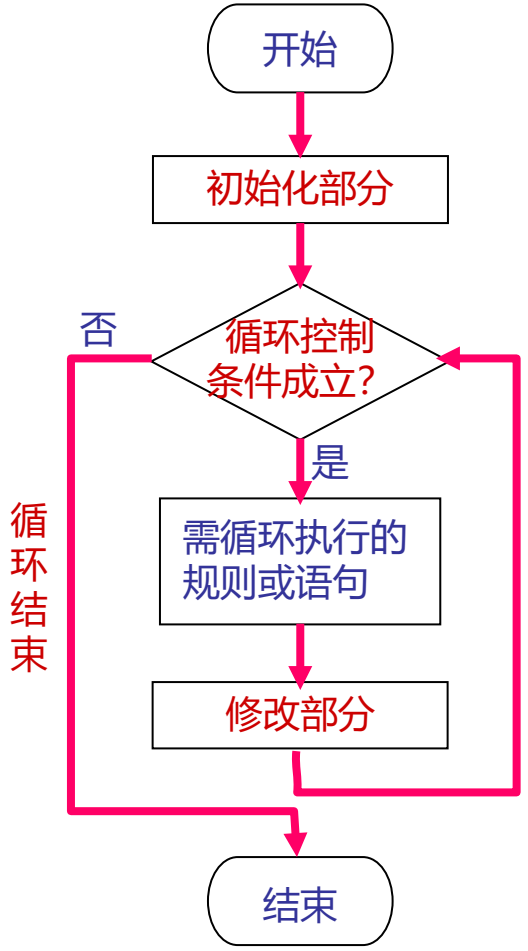
循环结构的流程图

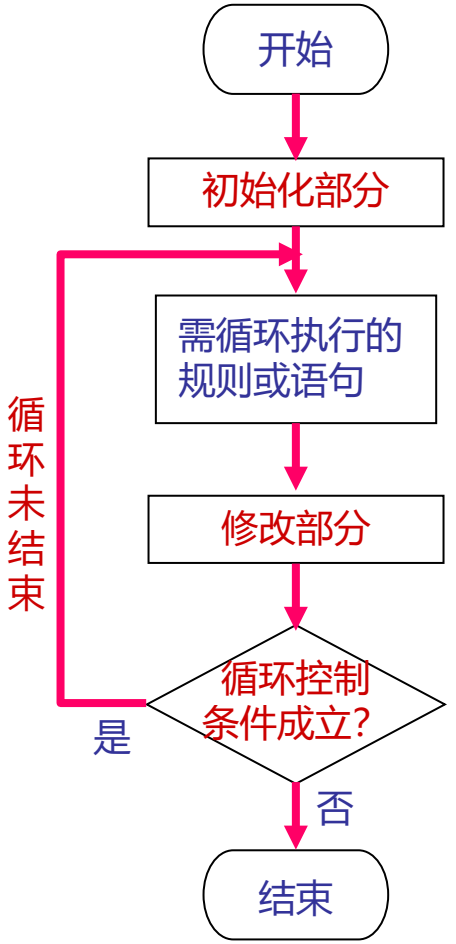
算法的表达-数据结构与控制结构

算法的程序流程图表达方法

循环结构的两种情况的流程图表达

有界循环结构的流程图(循环次数确定)





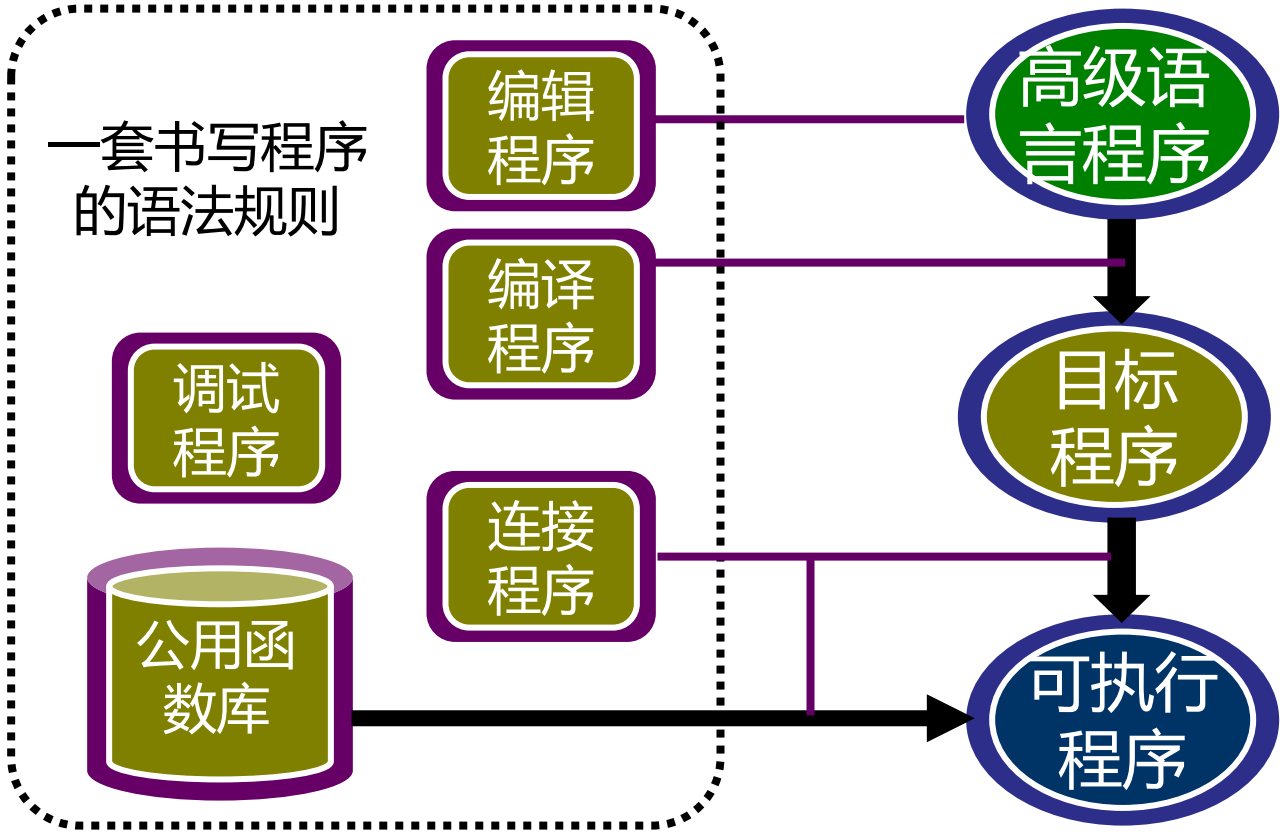
条件循环结构的流程图(循环次数不确定)

算法的表达-数据结构与控制结构

由算法到程序

程序设计过程：编辑源程序➔编译➔链接➔执行。

计算机语言程序设计环境：编辑、编译、连接、调试、运行一体化平台



TSP问题：算法的表达

求解TSP问题的遍历算法的表达

S	S[1]	S[2]	S[3]	S[4]
	1	2	3	4
	1	2	4	3
	1	3	2	4
	1	3	4	2
	.....			

Distance =
D[S[4]][S[1]] +
$\sum$ D[S[i]][S[i+1]]
i=1 to 3

当前最短路径 MS

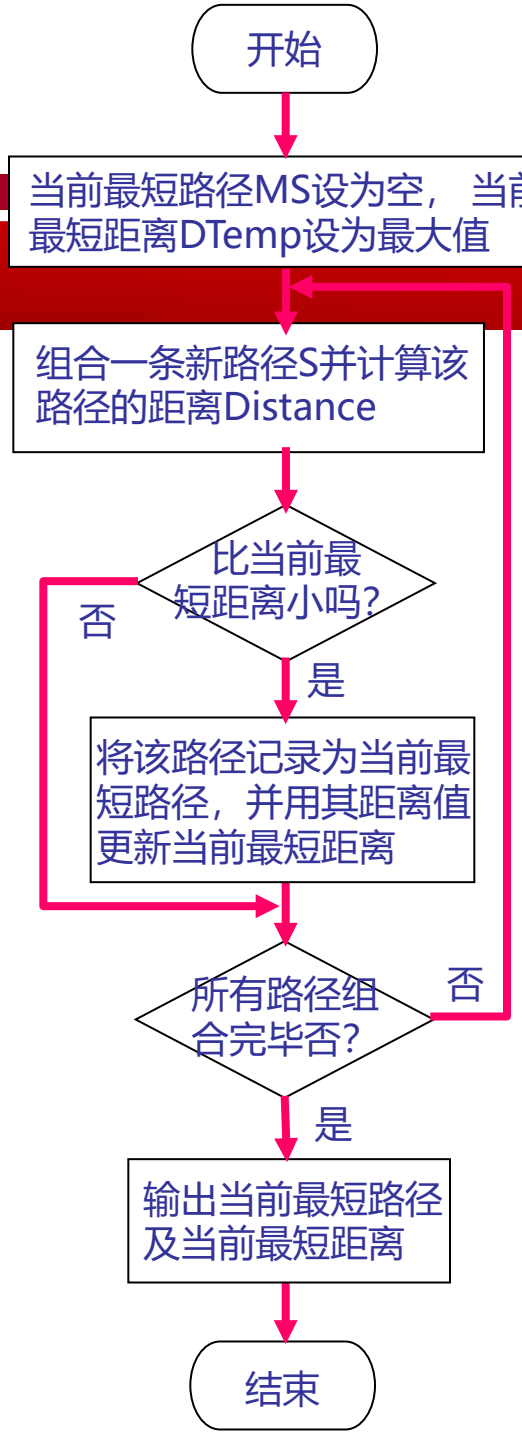
1	2	3	4

当前最短距离 DTemp

D

城市编号	1	2	3	4
1		2	6	5
2	2		4	4
3	6	4		2
4	5	4	2	

将思想/策略转变为“步骤”



TSP问题：算法的表达

求解TSP问题的贪心算法的表达

✓依次访问过的城市编号被记录在S[1], S[2], …, S[N}中,即第I 次访问的城市记录在S[I]中。

✓Step(1)：从1号城市开始访问，将城市编号1赋值给S[1]。

✓Step(6)：第I次访问的城市为城市j，其距第I-1次访问的城市的距离最短。

S

S[1]	S[2]	S[3]	S[4]
1			

I = 2, 3, 4 ---当前要找第几个

j = 1, 2, 3, 4 ---城市号

D

城市编号	1	2	3	4
1		2	6	5
2	2		4	4
3	6	4		2
4	5	4	2	

Start of the Algorithm

(1) S[1]=1;

(2) Sum=0;

(3) 初始化距离数组D[N, N];

(4) I=2;

(5) 从所有未访问过的城市中查找距离S[I-1]最近的城市j;

(6) S[I]=j;

(7) I=I+1;

(8) Sum=Sum+Dtemp;

(9) 如果I<=N，转步骤(5)，否则，转步骤(10);

(10)Sum=Sum+D[1, j];

(11)逐个输出S[N]中的全部元素;

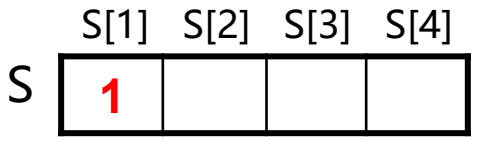
(12)输出Sum。

End of the Algorithm

TSP问题：算法的表达

求解TSP问题的贪心算法的表达

前述第5步“从所有未访问过的城市中查找距离S[I-1]最近的城市j”还是不够明确，需要进一步细化



I = 2, 3, 4 ---当前要找第几个

L = 1, 2, ...,I-1 ---从第1个访问过的，到第I-1个访问过的

K = 2, 3, 4 ---城市号

(5.1)K=2;
(5.2)将Dtemp设为一个大数(比所有两个城市之间的距离都大)
(5.3)L=1;
(5.4)如果S[L]==K，转步骤5.8；//该城市已出现过，跳过
(5.5)L=L+1;
(5.6)如果L<I，转5.4;
(5.7)如果D[K,S[I-1]]<Dtemp,则j=K; Dtemp = D[K,S[I-1]];
(5.8)K=K+1;
(5.9)如果K<=N，转步骤5.3。

TSP问题：算法的表达

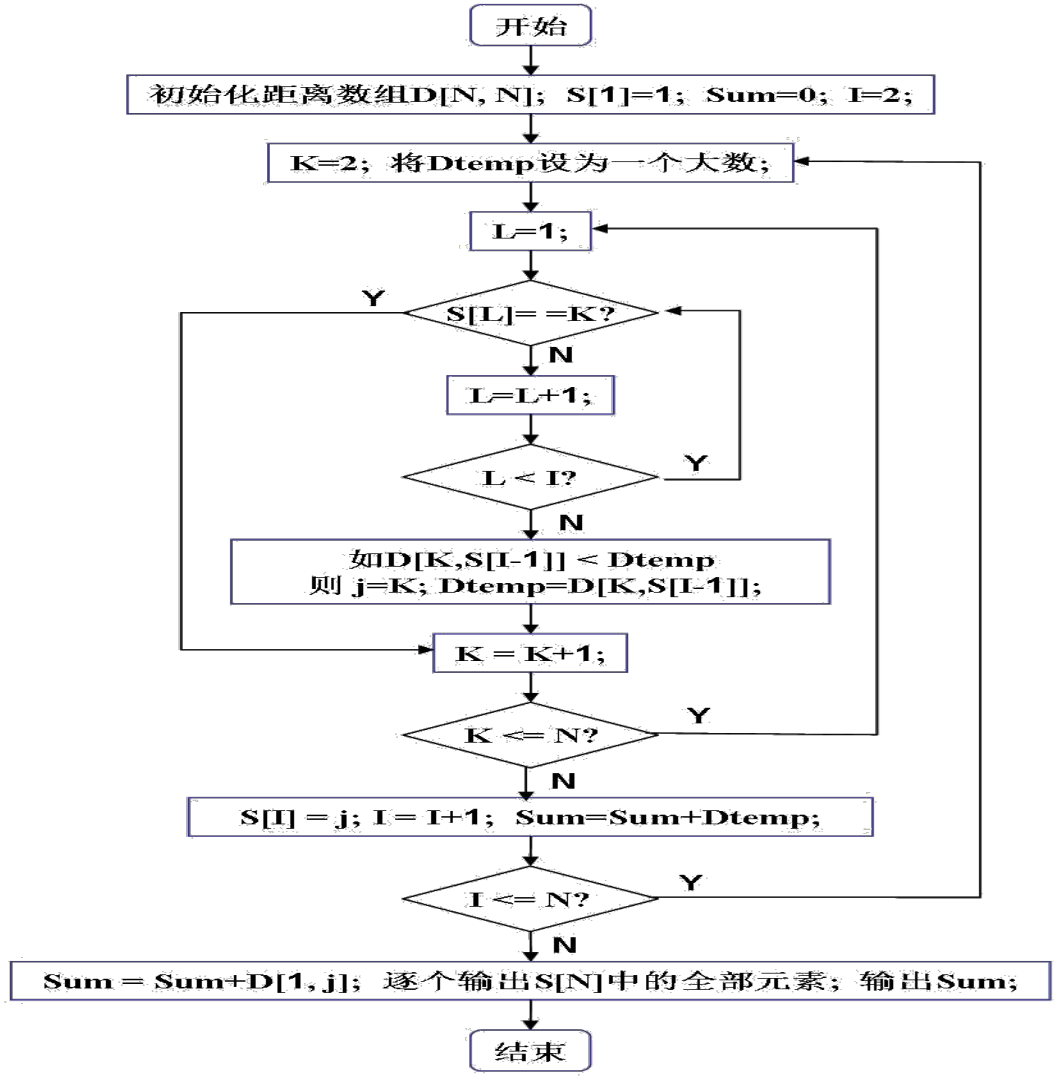
求解TSP问题的贪心算法的表达

	S[1]	S[2]	S[3]	S[4]
S	1			

I = 2, 3, 4 ---当前要找第几个

K = 2, 3, 4 ---城市号

L = 1, 2, ..., I-1 ---从第1个访问过的，到第I-1个访问过的



TSP问题：算法的表达

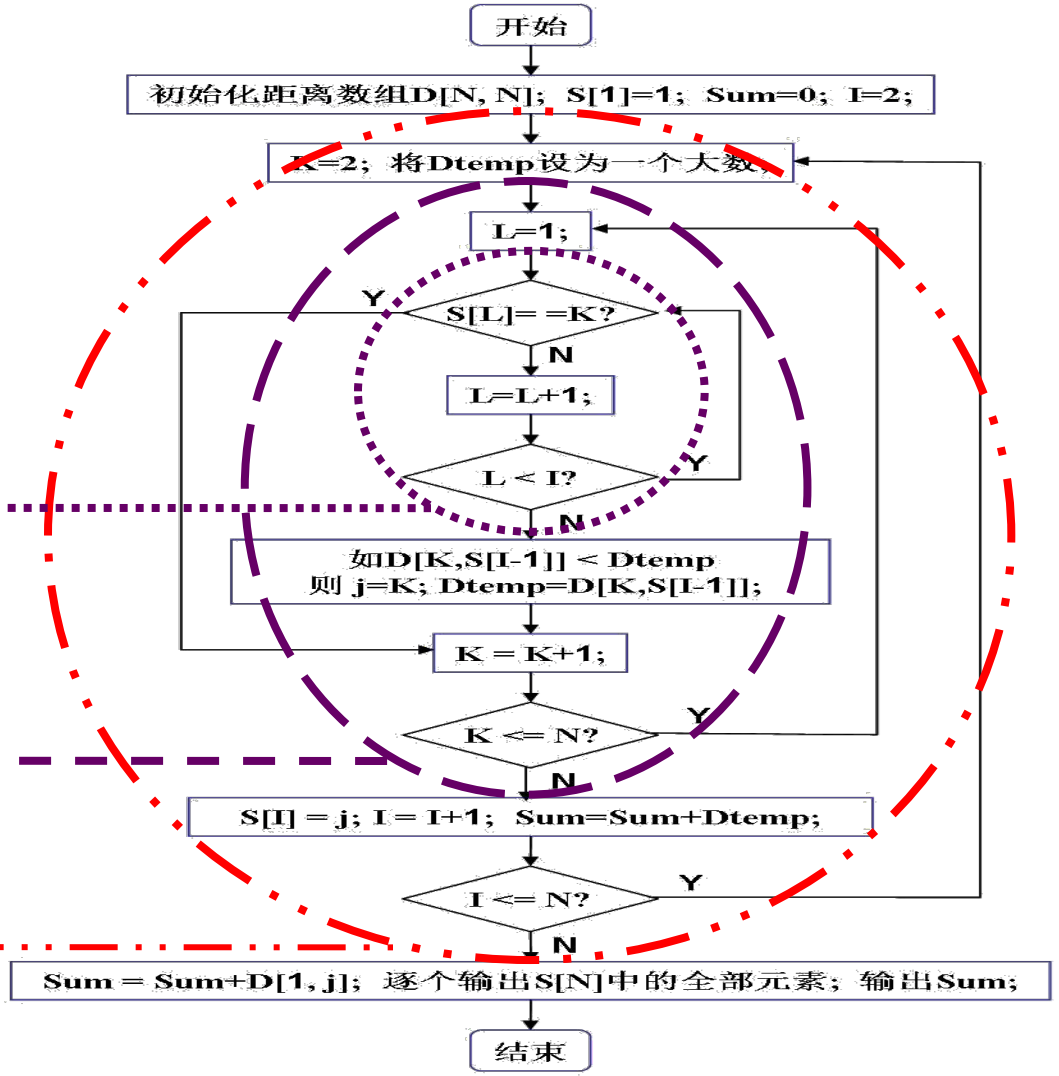
阅读并理解算法的能力

◆计算学科的学生不仅能够设计算法，而且会描述和表达算法，更要能读懂算法。

内层循环，L从1至I-1，循环判断第K号城市是否是已访问过的城市，如是则不参加最小距离的比较；

中层循环，K从2号城市至N号城市循环，判断D[K, S[I-1]]是否是最小值，j记录了最小距离的城市号K。

外层循环，I从2至N循环；I-1个城市已访问过，正在找与第I-1个城市最近距离的城市；已访问过的城市号存储在S[]中。

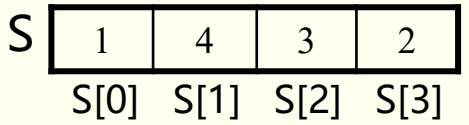


TSP问题：算法的表达

实现算法的程序：阅读与理解

```
main() {    判断城市K是否是已访问过的城市
…
//前面已为 K 和 I 赋过值
L=0;    Found=0;
do {
        if(S[L]==K)
        { Found=1; break; }
        else L=L+1;
    } while(L<I);       //L从1到I循环
if Found==0 { //城市K未出现过 }
else { //城市K出现过 }
…
}
```

- K从1,…,n-1是城市的编号
- I是至今已访问过的城市数
- S[0]至S[I-1]中存储的是已访问过的城市的编号

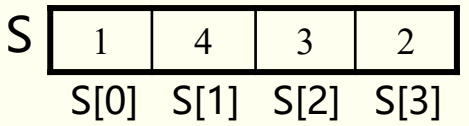


TSP问题：算法的表达

实现算法的程序：阅读与理解

```
main() {   ...    寻找下一个未访问过的城市j,且其距当前访问城市S[I-1]距离最短
    K=1;   Dtemp=10000;
    do{  // K从1到n-1循环
        L=0;   Found=0;
        do{
            if(S[L]==K)
            {   Found=1;   break;   }
            else L=L+1;
         } while(L<I);    //L从1到I循环
        if (Found==0 && D[K][S[I-1]]<Dtemp)
         {   j=K;   Dtemp=D[K][S[I-1]];   }
        K=K+1;
    } while(K<n);                    //K从1到n-1循环
    S[I]=j; Sum=Sum+Dtemp;
    ... }
```

- K从1,...,n-1是城市的编号
- I是至今已访问过的城市数
- S[I-1]中存储的是当前访问过的城市编号，要找第I个程序



- D[K][S[I-1]]为城市K距当前访问过的城市的距离

TSP问题：算法的表达

实现算法的程序：阅读与理解

TSP问题的贪心算法C语言程序实例

```
#include <stdio.h>
#define   n   4
main(){
   int     D[n][n], S[n], Sum, I, j, K, L, Dtemp, Found;
   S[0]=0;   Sum=0;     D[0][1]=2;   D[0][2]=6;   D[0][3]=5;   D[1][0]=2;   D[1][2]=4;
   D[1][3]=4;   D[2][0]=6;   D[2][1]=4;   D[2][3]=2;   D[3][0]=5;   D[3][1]=4;   D[3][2]=2;
   I=1;                    //注意程序是从 0 开始，流程图是从 1 开始的，下同.
   do {                    //I 从 1 到 n-1 循环
        K=1;   Dtemp=10000;
        do{                          //K 从 1 到 n-1 循环
             L=0;   Found=0;
             do{                          //L 从 1 到 I 循环
                  if(S[L]==K)
                  {   Found=1;   break;   }
                  else L=L+1;
               } while(L<I);          //L 从 1 到 I 循环
             if(Found==0 && D[K][S[I-1]]<Dtemp)
             {   j=K;   Dtemp=D[K][S[I-1]];   }
             K=K+1;
        } while(K<n);              //K 从 1 到 n-1 循环
        S[I]=j;   I=I+1;   Sum=Sum+Dtemp;
     } while(I<n);          //I 从 1 到 n-1 循环
   Sum=Sum+D[1][j];
   for(j=0;j<n;j++){   printf("%d,",S[j]);   }  //输出城市编号序列
   printf("\n");                                 //换行
   printf("Total Length:%d",Sum);                //输出总距离
}
```

TSP问题：算法的表达

实现算法的程序：阅读与理解

TSP 问题的贪心算法程序

Python实例

```
def greedy():
    """c语言转python版本"""
    matrix = [  [0.0,   19.02,  34.01,  44.10,  2684,   50.69],
                [19.02, 0.0,    2684,   45.88,  42.57,  2684],
                [34.01, 2684,   0.0,    2684,   54.33,  39.35],
                [44.10, 45.88,  2684,   0.0,    44.40,  11.18],
                [2684,  42.57,  54.33,  44.40,  0.0,    54.56],
                [50.69, 2684,   39.35,  11.18,  54.56,  0.0]]

    S = [0] * len(matrix)    # 有n个0的数组
    n = 6
    I = 1
    j = 0
    K = 0
    L = 0
    Dtemp = 0
    Found = 0
    Sum = 0
    while I < n:
        # 找第I次的最短访问节点
        K = 1
        Dtemp = 10000
        while K < n:
            # 假设第I次访问的是K，看K是不是已经访问过。。
            L = 0
            Found = 0
            while L < I:
                if S[L] == K:
                    Found = 1
                    break
                L += 1
            if Found == 0 and matrix[K][S[I-1]] < Dtemp:
                j = K
                Dtemp = matrix[K][S[I-1]]
            K += 1
        S[I] = j
        I += 1
        Sum += Dtemp
    Sum += matrix[0][j]
    S.append(0)
    for j in range(len(S)): print("%d\t" % S[j], end='')
    print('\n总距离%.2f' % Sum)
```

第7讲 算法：程序与计算系统之灵魂

一、算法概述

二、问题与数学建模

三、算法策略

四、算法的表达-数据结构与控制结构

五、算法复杂性与计算复杂性

【示例】

TSP问题：问题与数学建模

TSP问题：算法策略

TSP问题：算法的表达

TSP问题：算法复杂性与计算复杂性

算法复杂性与计算复杂性

算法的模拟与分析

算法是正确的吗?

◆算法的正确性问题

- ✓ 问题求解的过程、方法——算法是正确的吗？算法的输出是问题的解吗？满足问题求解约束的解就是问题的解，但不一定是最优解。
- ✓ 20世纪60年代，美国一架发往金星的航天飞机由于控制程序出错而永久丢失在太空中。

◆ 算法的效果评价

- ✓ 算法的输出是最优解还是可行解？如果是可行解，与最优解的偏差多大？

■ 两种评价方法：

- ✓ 证明方法：利用数学方法证明；
- ✓ 仿真分析方法：产生或选取大量的、具有代表性的问题实例，利用该算法对这些问题实例进行求解，并对算法产生的结果进行统计分析。

算法获得的解是最优的吗?

TSP问题：算法复杂性与计算复杂性

TSP问题贪心算法的正确性评价

直观上只需检查算法的输出结果中，每个城市出现且仅出现一次，该结果即是TSP问题的可行解，说明算法正确地求解了这些问题实例

	S[1]	S[2]	S[3]	S[4]
S	1	2	3	4

```
#include <stdio.h>
#define  n  4
main(){
  int   D[n][n], S[n], Sum, I, j, K, L, Dtemp, Found;
  S[0]=0;  Sum=0;   D[0][1]=2;  D[0][2]=6;  D[0][3]=5;  D[1][0]=2;  D[1][2]=4;
  D[1][3]=4;  D[2][0]=6;  D[2][1]=4;  D[2][3]=2;  D[3][0]=5;  D[3][1]=4;  D[3][2]=2;
  I=1;                    //注意程序是从 0 开始，流程图是从 1 开始的，下同.
  do {                    //I 从 1 到 n-1 循环
     K=1;  Dtemp=10000;
     do{                          //K 从 1 到 n-1 循环
        L=0;  Found=0;
        do{                       //L 从 0 到 I-1 循环
           if(S[L]==K)
           {  Found=1;  break;  }
           else L=L+1;
        } while(L<I);             //L 从 0 到 I-1 循环
        if(Found==0 && D[K][S[I-1]]<Dtemp)
        {  j=K;  Dtemp=D[K][S[I-1]];  }
        K=K+1;
     } while(K<n);                //K 从 1 到 n-1 循环
     S[I]=j;  I=I+1;  Sum=Sum+Dtemp;
  } while(I<n);                   //I 从 1 到 n-1 循环
  Sum=Sum+D[j][0];
  for(j=0;j<n;j++){  printf("%d,",S[j]);  }  //输出城市编号序列
  printf("\n");                               //换行
  printf("Total Length:%d",Sum);              //输出总距离
}
```

TSP问题：算法复杂性与计算复杂性

TSP问题贪心算法的效果评价

✓如果实例的最优解已知（问题规模小或问题已被成功求解），利用统计方法对若干问题实例的算法结果与最优解进行对比分析，即可对其进行效果评价；

✓对于较大规模的问题实例，其最优解往往是未知的，因此，算法的效果评价只能借助于与前人算法结果的比较。

	S[1]	S[2]	S[3]	S[4]
S	1	2	3	4

```
#include <stdio.h>
#define  n   4
main(){
  int    D[n][n], S[n], Sum, I, j, K, L, Dtemp, Found;
  S[0]=0;  Sum=0;   D[0][1]=2;  D[0][2]=6;  D[0][3]=5;  D[1][0]=2;  D[1][2]=4;
  D[1][3]=4;  D[2][0]=6;  D[2][1]=4;  D[2][3]=2;  D[3][0]=5;  D[3][1]=4;  D[3][2]=2;
  I=1;                    //注意程序是从 0 开始，流程图是从 1 开始的，下同.
  do {                    //I 从 1 到 n-1 循环
     K=1;  Dtemp=10000;
     do{                  //K 从 1 到 n-1 循环
        L=0;  Found=0;
        do{               //L 从 0 到 I-1 循环
           if(S[L]==K)
           {  Found=1;  break;  }
           else L=L+1;
        } while(L<I);     //L 从 0 到 I-1 循环
        if(Found==0 && D[K][S[I-1]]<Dtemp)
        {  j=K;  Dtemp=D[K][S[I-1]];  }
        K=K+1;
     } while(K<n);        //K 从 1 到 n-1 循环
     S[I]=j;  I=I+1;  Sum=Sum+Dtemp;
  } while(I<n);           //I 从 1 到 n-1 循环
  Sum=Sum+D[j][0];
  for(j=0;j<n;j++){  printf("%d,",S[j]);  }  //输出城市编号序列
  printf("\n");                              //换行
  printf("Total Length:%d",Sum);             //输出总距离
}
```

算法复杂性与计算复杂性

算法的时间复杂度与空间复杂度

算法获得结果的时间有多长?

◆**时间复杂性:** 如果一个问题的规模是n，解这一问题的某一算法所需要的时间为T(n)，它是n的某一函数，则T(n)称为这一算法的“时间复杂性”。

◆ **“大O记法”：**

✓基本参数 n——问题实例的规模

✓把复杂性或运行时间表达为n的函数。

✓ “O”表示量级 (order)，允许使用“=”代替“≈”，如$n^2+n+1 = O(n^2)$。

◆**空间复杂性:** 算法在执行过程中所占存储空间的大小。



算法复杂性与计算复杂性

算法复杂性分析示例

```
sum=0;                                  (1次)
for( i=1; i<=n; i++)                    (n次)
{  for( j=1; j<=n; j++)                 (n²次)
        { sum++;  }                     (n²次)
}
```

解：$T(n) = 2n^2+n+1 = O(n^2)$

主要关注点：循环的层数、递归/迭代的次数

算法复杂性与计算复杂性

不同的算法复杂性量级

问题规模n	计算量
10	10!
20	20!
100	100!
1000	1000!
10000	10000!

$20! = 1.216 \times 10^{17}$

$20^3 = 8000$

$O(n^3)$与$O(3^n)$的差别，$O(n!)$与$O(n^3)$的差别

$O(n^3)$	$O(3^n)$
0.2秒	4×10^{28}秒 =1015年
注：每秒百万次，n=60，1015年相当于10亿台计算机计算一百万年	

$O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^b)$

$O(b^n)$, $O(n!)$

算法复杂性与计算复杂性

有限时间内是否能完成算法的执行?

◆当算法的复杂度是一个多项式，如$O(n^2)$时，则对于大规模问题，该算法是可以在有限时间内被计算机完成的。例如，TSP问题的贪心算法 $O(n^3)$ 。

$O(1), O(\log n), O(n), O(n \log n), O(n^b)$

◆当算法的复杂度是用指数函数表示如$O(2^n)$或**阶乘函数**表示如$O(n!)$，则当n很大（如10000）时，该算法是计算机在有限时间内无法处理的。例如TSP问题的遍历算法O((n-1)!)。

$O(b^n), O(n!)$

单选题 1分

设置

71

测练题：右侧算法的复杂性是_____？

A $O(n^5)$

B $O(n^5+n^4)$

C $O(n^2)$

D $O(1)$

```
For A=1 to 9 Step 1
{  For B=1 to 9 Step 1
   {  For C=1 to 9 Step 1
      {  For D=1 to 9 Step 1
         {  For E=1 to 9 Step 1
            {  if (  ((A*10+B) %2 ==0) &&
                     ((A*100+B*10+C) %3==0) &&
                     ((A*1000+B*100+C*10+D) %4==0) &&
                     ((A*10000+B*1000+C*100+D*10+E) %5==0) &&
                     A!=B && A!=C && A!=D && A!=E &&
                     B!=C && B!=D && B!=E &&
                     C!=D && C!=E && D!=E )
               { 输出"ABCDE"}
            }}}}}
```

提交

TSP问题：算法复杂性与计算复杂性

TSP问题贪心算法的复杂性分析

✓一个关于n的三重循环。

✓时间复杂度是$O(n^3)$。

```
#include <stdio.h>
#define  n  4
main(){
  int    D[n][n], S[n], Sum, I, j, K, L, Dtemp, Found;
   S[0]=0;  Sum=0;   D[0][1]=2;  D[0][2]=6;  D[0][3]=5;  D[1][0]=2;  D[1][2]=4;
   D[1][3]=4;  D[2][0]=6;  D[2][1]=4;  D[2][3]=2;  D[3][0]=5;  D[3][1]=4;  D[3][2]=2;
   I=1;                    //注意程序是从 0 开始，流程图是从 1 开始的，下同.
  do {                     //I 从 1 到 n-1 循环---将被执行 n-1 次，简记约 n 次
      K=1;  Dtemp=10000;
      do{                  //K 从 1 到 n-1 循环---将被执行(n-1)*(n-1)次，简记约
          L=0;  Found=0;
          do{              //L 从 1 到 I 循环---将被执行，简
              if(S[L]==K)
              {  Found=1;  break;  }
              else L++;
          } while(L<I);    //L 从 1 到 I 循环
          if(Found==0 && D[K][S[I-1]]<Dtemp)
          {  j=K;  Dtemp=D[K][S[I-1]];  }
          K++;
      } while(K<n);        //K 从 1 到 n-1 循环
      S[I]=j;  I=I+1;  Sum=Sum+Dtemp;
   } while(I<n);           //I 从 1 到 n-1 循环
   Sum=Sum+D[1][j];
   for(j=0;j<n;j++){  printf("%d,",S[j]);  }  //输出城市编号序列
   printf("\n");                              //换行
   printf("Total Length:%d",Sum);             //输出总距离
}
```

n

n^2

n^3

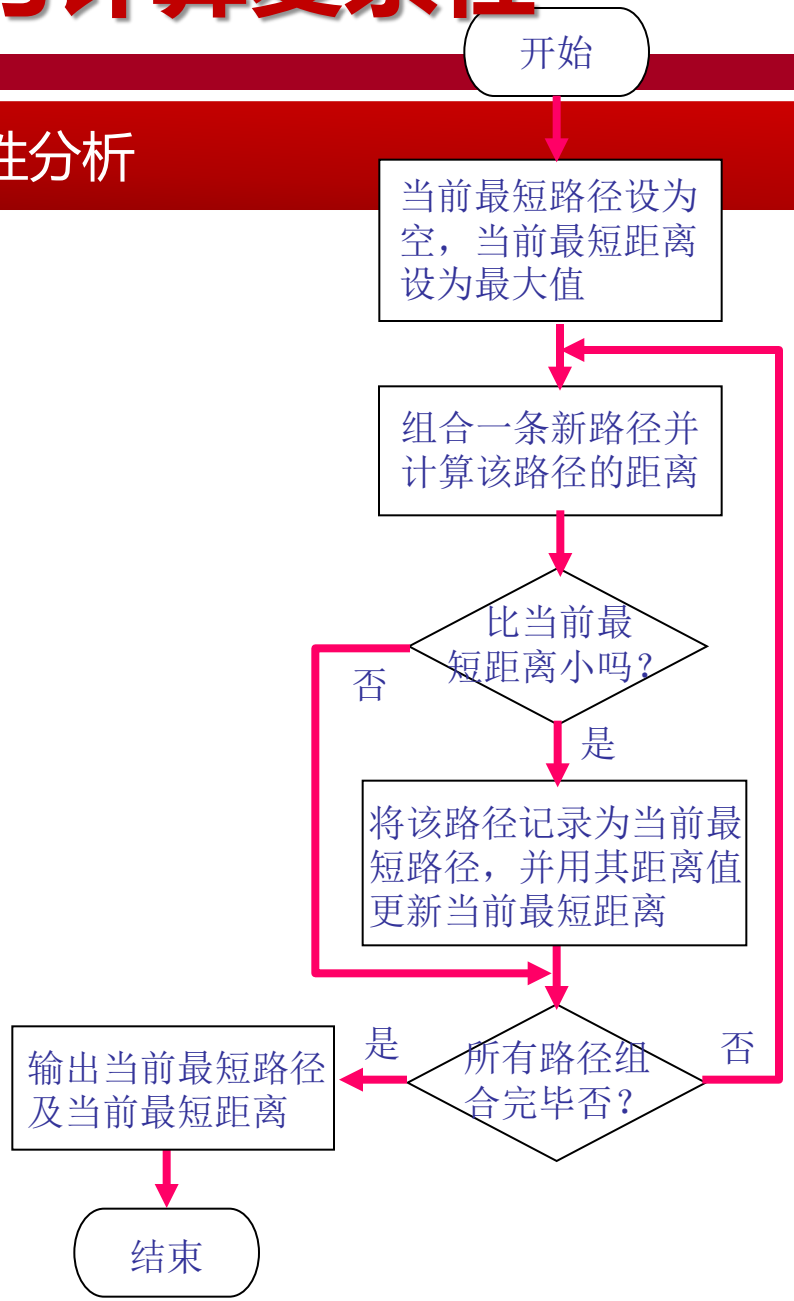
TSP问题：算法复杂性与计算复杂性

TSP问题遍历算法的复杂性分析

✓列出每一条可供选择的路线，计算出每条路线的总里程，最后从中选出一条最短的路线。

✓路径组合数目为(n-1)!

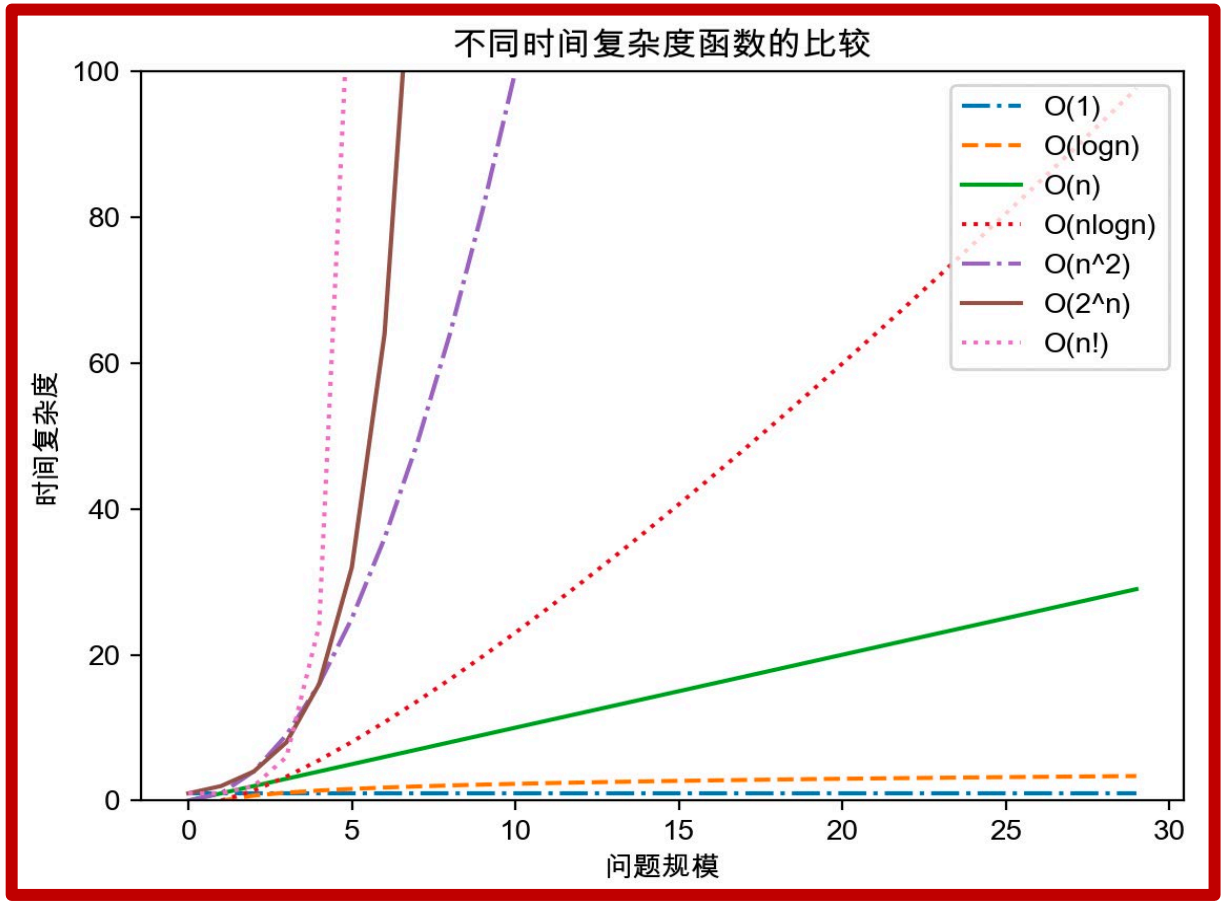
✓时间复杂度是O((n-1)!)



算法复杂性

不同量级复杂性在计算时间方面的差异

时间复杂度	问题规模		
	10	100	1000
O(1)	0.001 毫秒	0.001 毫秒	0.001 毫秒
O(logn)	0.003 毫秒	0.006 毫秒	0.0096 毫秒
O(n)	0.01 毫秒	0.1 毫秒	1 毫秒
O(nlogn)	0.03 毫秒	0.66 毫秒	9.97 毫秒
$O(n^2)$	0.1 毫秒	10 毫秒	1 秒
$O(2^n)$	1 毫秒	$4*10^{16}$ 年	$3.4*10^{287}$ 年
O(n!)	3.6 秒	$2.9*10^{144}$ 年	$1.3*10^{2555}$ 年



算法分析

有限时间内是否能完成算法的执行?

◆当算法的复杂度是一个多项式，如$O(n^2)$时，则对于大规模问题，该算法是可以在有限时间内被计算机完成的。例如，TSP问题的贪心算法 $O(n^3)$ 。

$O(1), O(\log n), O(n), O(n \log n), O(n^b)$

◆当算法的复杂度是用指数函数表示如$O(2^n)$或阶乘函数表示如$O(n!)$，则当n很大（如10000）时，该算法是计算机在有限时间内无法处理的。例如TSP问题的遍历算法O((n-1)!)。

$O(b^n), O(n!)$

算法分析

计算复杂性与算法复杂性

◆**算法复杂性：** 求解问题的某一个算法的复杂性。

◆**计算复杂性：** 能求问题精确解的那个算法的复杂性。

计算复杂性是指问题的一种特性，即利用计算机求解问题的难易性或难易程度

问题的计算复杂性

- 计算机在有限时间内能够求解的--【**可求解问题**】
- 计算机在有限时间内不能求解的--【**难求解问题**】
- 计算机完全不能求解的--【**不可计算问题**】

算法分析【示例】TSP问题

TSP问题的计算复杂性与算法复杂性

- TSP问题的遍历算法的时间复杂度：$O((n-1)!)$
- TSP问题的贪心算法的时间复杂度：$O(n^3)$。

算法复杂性
-某一个算法的复杂性

- TSP问题的计算复杂度：$O((n-1)!)$

计算复杂性
-求精确解的那个算法的复杂性

TSP问题的遍历算法（又称穷举或蛮干）

对解空间中的每一个可能解进行验证，直到所有的解都被验证是否正确，便能得到精确的结果---**精确解**

可能是$O(n!)$或$O(a^n)$

TSP问题的贪心算法

近似解求解算法---**近似解**

应该是$O(n^a)$

Δ = | 近似解 － 精确解 |
满意解： Δ充分小时的近似解

算法复杂性与计算复杂性

证比求易问题

问题的引入——国王求婚 问题

求出48 770 428 433 377 171 （17位数）的一个真因子。

算法1：从2开始逐个除，一秒一个，一天算86400个

答案：223 092 827。

算法2：最小的真因子不会超过9位，全民动员

串行与并行，时间与空间，折衷

算法复杂性与计算复杂性

可解性问题与难解性问题

■P类问题：多项式问题(Polynomial Problem)，指计算机可以在有限时间内求解的问题，即：可以找出一个呈现$O(n^a)$复杂性算法求出精确解的问题，其中a为常数。

■NP类问题：非确定性多项式问题(Non-deterministic Polynomial)。有些问题，无法直接求精确解，只能通过猜算或试算来得到结果(枚举-计算-验证)，这就是非确定性问题。对该类问题，虽然在多项式时间内难于求解但不难判断给定一个解的正确性，即：在多项式时间内可以由一个算法验证一个解是否正确的非确定性问题，就是NP类问题。

■NPC类问题：对NP类问题，如果（1）如果NP问题的所有可能答案都可以在多项式时间内进行正确与否的验算；（2）求精确解只能遍历每一个可能解；（3）遍历所有可能解的复杂性呈指数或阶乘级别，则该问题被称为完全非确定性多项式问题(NP-Complete)。



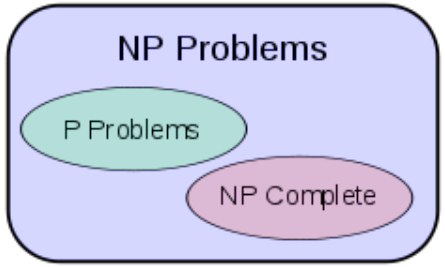
问：加密算法应设计成一个什么问题呢?

算法复杂性与计算复杂性

可解性问题与难解性问题

- 所有可以在多项式时间内求解的问题，称为【**P类问题**】
- 所有在多项式时间内可以验证的问题，称为【**NP类问题**】，P⊆NP。
- Open problem: P=NP? 美国克雷数学研究所百万大奖难题。







计算复杂性理论应用—密码学

- 私钥密码体制，加、解密规则一致，易破解
  - 明文、密钥、密文、加密算法、解密算法
- 公钥密码系统 RSA——提出者获得图灵奖
  - 利用加密秘钥→解密密钥 是一个NP Complete问题，在有效时间不可能求解，本质是大合数的真因子分解
  - 密码体系建立在NP Complete问题上，若NP Complete能在多项式时间内求解，则结果是可怕的

计算复杂性理论应用—密码学

公钥密码系统 RSA

加密密钥公开，解密密钥私有

例：三月二十八日早晨七点不发起总攻

三月二十八日早晨七点不发起总攻
解密密钥：质数位无效。
嵌入了干扰字符，可利用猜、统计、数学方法等解密。

算法：程序与计算系统之灵魂

小结：理解算法类问题求解框架

